Gemini 3.8 Flash Latency & Architecture Evaluation Report

Empirical Analysis of Streaming Delivery, `ThinkingConfig`, Pydantic Adaptive Routing & 50-Turn Context Caching on Google AI Studio (`google-genai` SDK)

Author: Yuichi Ito | **Date:** 2026-11 | **Target:** Google AI Studio (`google-genai` SDK / `gemini-3.8-flash`)

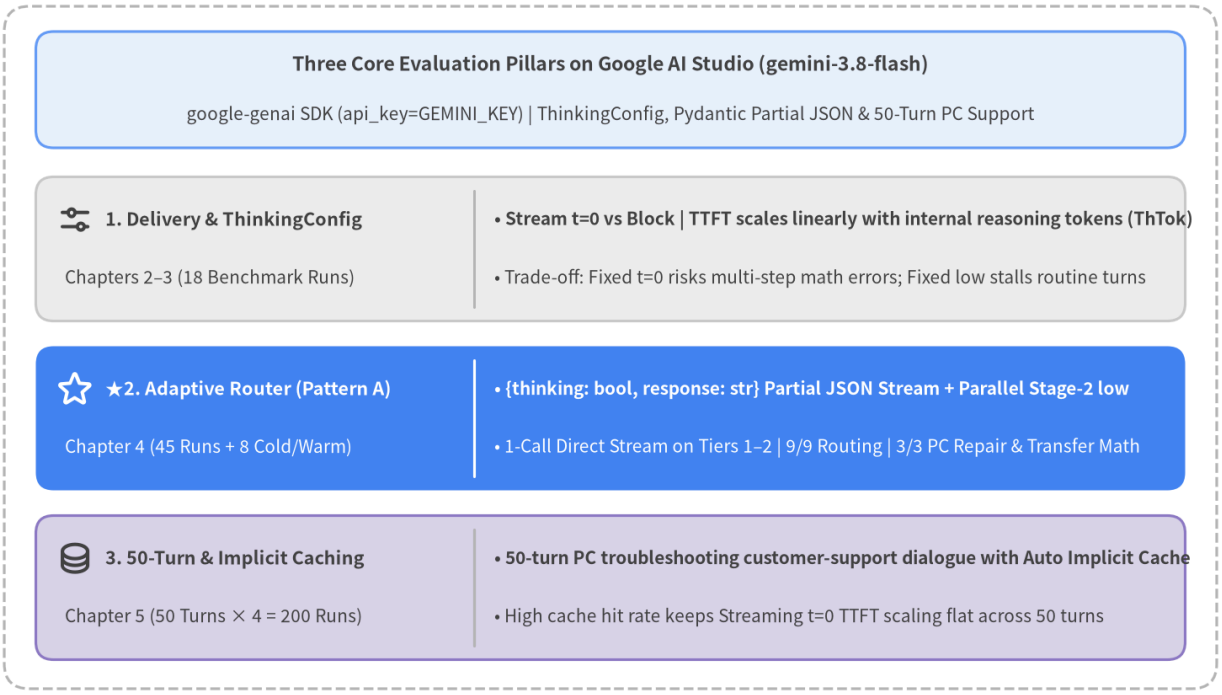


Figure 0-1: Three Core Evaluation Pillars and Key Empirical Findings on Google AI Studio (gemini-3.8-flash)

Report Overview & Chapter Structure (Chapters 1–5)

This technical report presents a comprehensive empirical evaluation of `gemini-3.8-flash` on **Google AI Studio (`google-genai` SDK via `genai.Client(api_key=GEMINI_KEY)`)**, synthesizing **271 live API benchmark executions** recorded in `src/gemini_evaluation/` under a PC

troubleshooting customer-support scenario. It establishes actionable architecture patterns that simultaneously achieve responsive Time to First Token (TTFT), multi-step mathematical accuracy, and flat latency scaling across 50-turn conversational histories.

Chapter	Title	Scope & Key Empirical Findings
Chapter 1	**Executive Summary**	End-to-end architecture decision matrix, cross-benchmark latency comparisons, and production configuration guidelines.
Chapter 2	**Background & Measurement Methodology**	Anatomy of the 3 LLM latency phases (Prefill, Thinking, Decoding), 2D trajectory charts (`Elapsed Time vs. Cumulative Characters`), Pydantic `response_schema` Cold vs. Warm FSM grammar compilation, incremental delta extraction via `pydantic_core.from_json(..., allow_partial="trailing-strings")` , and `ExecutionTracer` instrumentation.
Chapter 3	**Parameters & Latency**	Empirical evaluation of Blocking (`generate_content`) vs. Streaming (`generate_content_stream`) across all 6 `ThinkingConfig` levels (`0` , `minimal` , `low` , `medium` , `high` , `default`), the linear scaling law of `thoughts_token_count` , and the speed-vs-accuracy trade-off across PC support query complexity Tiers 1–3.
Chapter 4	**Adaptive Thinking Router**	Evaluation of 3 adaptive routing architectures (Pattern A: 2-field Pydantic integrated stream, Pattern B: 1-field sequential router, Pattern C: plain-text tag prefix) across 45 runs. Demonstrates why **Pattern A** achieves `100%` routing accuracy and `100%` Tier 3 calculation accuracy with decoder-enforced JSON type safety.
Chapter 5	**Multi-Turn Chat Behavior & Context Caching**	50-turn continuous PC troubleshooting customer-support dialogue benchmark (200 total API calls). Analyzes zero-configuration **Implicit Context Caching** and flat TTFT scaling across long conversations.

Document Metadata	Value
Title (`title`)	Gemini 3.8 Flash Latency & Architecture Evaluation Report
Subtitle (`subtitle`)	Empirical Analysis of Streaming Delivery, ThinkingConfig, Pydantic Adaptive Routing & 50-Turn Context Caching
Author (`author`)	Yuichi Ito
Date (`date`)	2026-11
Target SDK & Model	Google AI Studio `google-genai` SDK (`genai.Client(api_key=GEMINI_KEY)`) / `gemini-3.8-flash`
Evaluation Codebase	`src/gemini_evaluation/llm_call.py` , `src/gemini_evaluation/adaptive_thinking.py` , `src/gemini_evaluation/tracer.py`

Table of Contents

1. Executive Summary

2. Background & Measurement Methodology

3. Parameters & Latency

4. Adaptive Thinking Router Pattern

5. Multi-Turn Chat Behavior & Context Caching

1. Executive Summary

This report presents an end-to-end empirical evaluation of `gemini-3.8-flash` on **Google AI Studio (`google-genai` SDK via `genai.Client(api_key=GEMINI_KEY)`)**, addressing how production conversational AI systems (specifically PC technical support and troubleshooting assistants) can simultaneously achieve three competing engineering goals: **(1) low-latency Time to First Token (`TTFT`), (2) `100%` accuracy on multi-step mathematical and conditional reasoning tasks (PC repair-vs-replacement cost break-even, SSD backup transfer time, and multi-PC warranty fee calculations), and (3) flat latency scaling across 50-turn multi-turn dialogues**. Synthesizing **271 live API benchmark executions** (`18` single-turn `ThinkingConfig` trials, `45` adaptive routing runs, `8` Pydantic schema Cold/Warm compilation trials, and `200` 50-turn multi-turn PC support chat executions), we establish **Streaming delivery (`generate_content_stream`) combined with the Pydantic 2-field Adaptive Thinking Router (`Pattern A`: `{thinking: bool, response: str}` parsed incrementally via `pydantic_core.from_json(..., allow_partial="trailing-strings")`) and prefix-immutable multi-turn history** as the recommended production architecture.

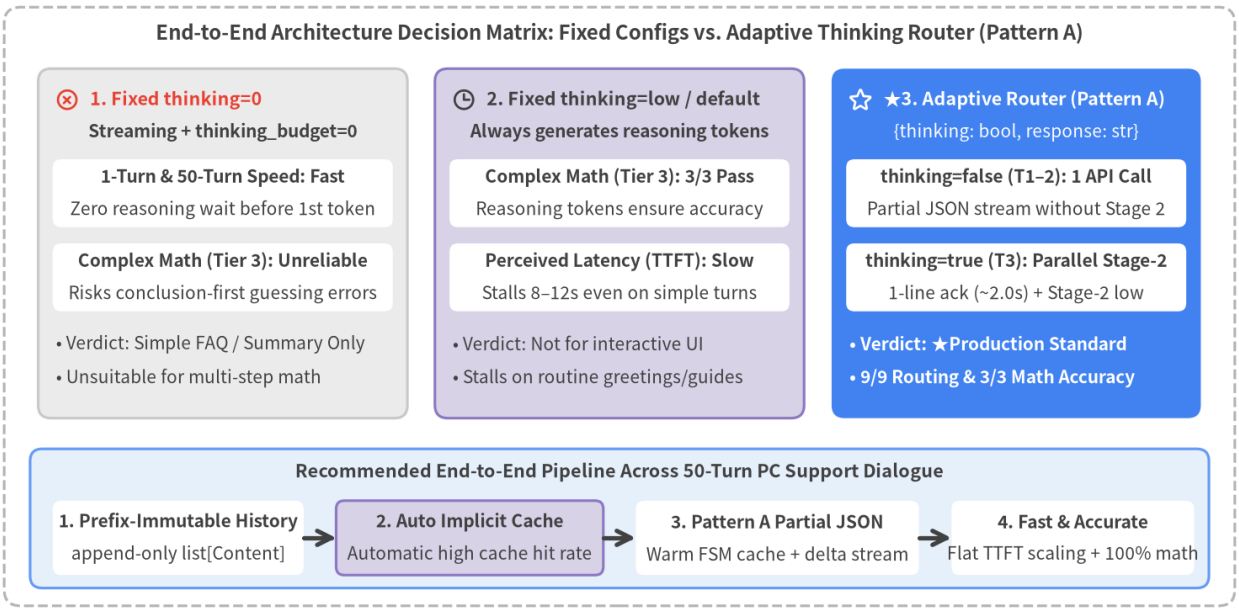


Figure 1-1: End-to-End Architecture Decision Matrix — Fixed Thinking Baselines vs. Adaptive Thinking Router (Pattern A) Across Single-Turn & 50-Turn Workloads

1.1 Cross-Benchmark Empirical Comparison

Figure 1-2 synthesizes the headline latency metrics—**Time to First Token (TTFT)** and **Total Completion Time (`stream_completed_ms`)**—across all three benchmark suites in this report: (1) Single-Turn Delivery Mode & `ThinkingConfig`, (2) Adaptive Thinking Router across Tiers 1–3, and (3) 50-Turn Multi-Turn PC Support Dialogue.

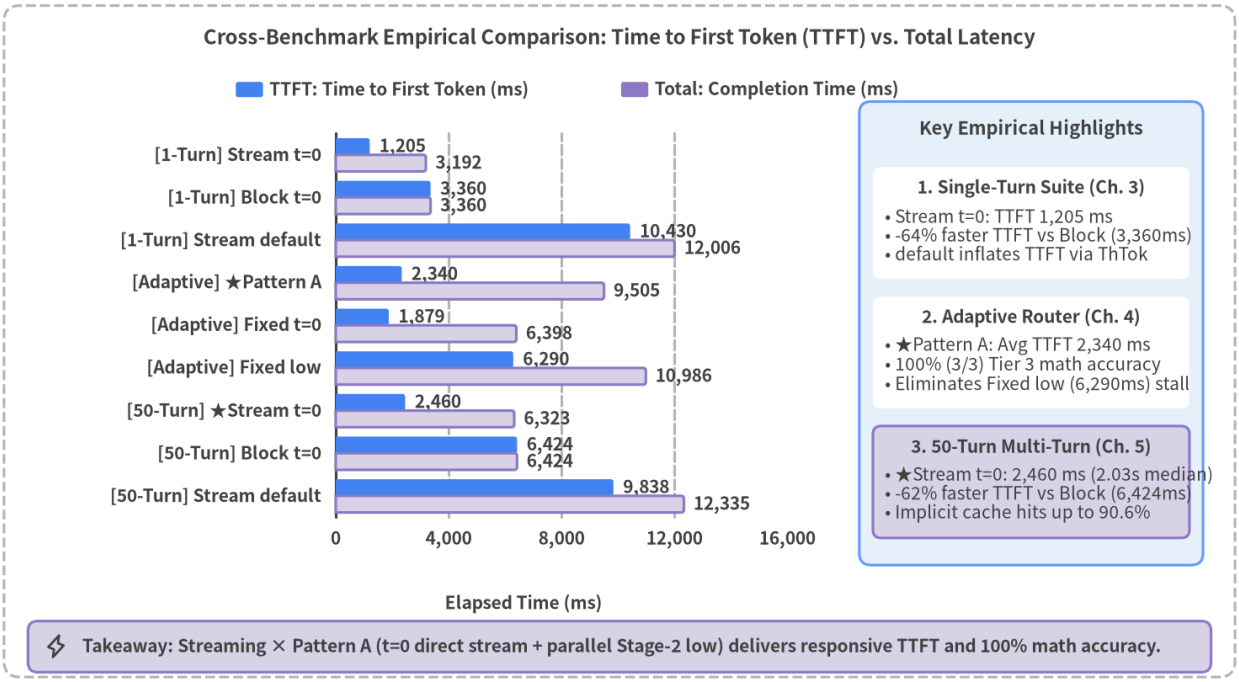


Figure 1-2: Cross-Benchmark Comparison of Time to First Token (TTFT) and Total Completion Time Across Single-Turn, Adaptive Router, and 50-Turn Multi-Turn Suites

Executive Summary Matrix Across All Benchmark Suites

Benchmark Suite & Workload	Architecture / Configuration	Avg TTFT	Avg Total	Reasoning (`ThTok`)	Accuracy & Scalability Profile	Verdict
Single-Turn (Ch. 3, PC boot diagnostic)	**Streaming** `thinking=0`	`1,205 ms`	`3,192 ms`	`0 tok`	`-64%` faster initial display vs. Blocking	**★Recommended (Simple)**
Single-Turn	Blocking `thinking=0`	`3,360 ms`	`3,360 ms`	`0 tok`	UI freezes for `3.36 s` until full completion	Batch Only

Benchmark Suite & Workload	Architecture / Configuration	Avg TTFT	Avg Total	Reasoning (ThTok)	Accuracy & Scalability Profile	Verdict
Single-Turn	Streaming `thinking=default`	`10,430 ms`	`12,006 ms`	`1,006 tok`	Unbounded reasoning adds heavy pre-token delay	Not Recommended
Adaptive Router (Ch. 4, Tiers 1–3, 45 runs)	**★Pattern A** (`{thinking, response}`)	`2,340 ms` (T3: `2,314 ms`)	`9,505 ms`	T1–2: `0` T3: `901 tok`	`9/9` routing & `3/3` **(100%)** Tier 3 math accuracy; FSM JSON type safety (`1.33` avg calls)	**★Production Standard**
Adaptive Router	Pattern C (`[DIRECT]` / `[THINK]` tag)	`1,879 ms`	`8,843 ms`	T1–2: `0` T3: `828 tok`	`9/9` routing & `3/3` math, but lacks schema safety and risks tag leakage	Reject (Unify on A)
Adaptive Router	Pattern B (`{thinking}` 1-field router)	`6,177 ms` (T3: `11,488 ms`)	`11,535 ms`	T1–2: `0` T3: `628 tok`	Always incurs `2.0` sequential API calls, stalling Tier 3 TTFT to `11.5 s`	Reject
Adaptive Router	Fixed `thinking=0` (`base_0`)	`1,879 ms`	`6,398 ms`	`0 tok`	**Autoregressive trap on Tier 3 math**—no internal scratchpad verification	Reject
Adaptive Router	Fixed `thinking=low` (`base_low`)	`6,290 ms` (T3: `9,289 ms`)	`10,986 ms`	Avg `439 tok`	`3/3` Tier 3 accuracy, but freezes UI up to `12.2 s` on routine queries	Not Recommended
50-Turn Multi-Turn (Ch. 5, PC support dialogue)	**★Streaming** `thinking=0`	`2,460 ms` (Median `2,029 ms`)	`6,323 ms`	`19 tok` (T2–50: `0`)	`-3,964 ms` (`-62%`) faster TTFT vs. Blocking; Implicit Cache hits up to `90.6%`	**★Recommended (Multi-Turn)**
50-Turn Multi-Turn	Blocking `thinking=0`	`6,424 ms`	`6,424 ms`	`0 tok`	Waits for full `~753-char` completion every turn	Batch Only
50-Turn Multi-Turn	Streaming `thinking=default`	`9,838 ms`	`12,335 ms`	Avg `858 tok`	Stream stalls for `~9.8s` until reasoning finishes	Not Recommended

1.2 Five Actionable Production Recommendations

Based on the empirical evidence across Chapters 2–5, engineering teams deploying `gemini-3.8-flash` via the Google AI Studio `google-genai` SDK should enforce the following five architectural practices:

1. **Always Use `generate_content_stream` for Interactive User Interfaces**: Streaming `thinking=0` reduces single-turn TTFT by `64%` (`1,205 ms` **vs.** `3,360 ms`) compared to Blocking, and in 50-turn multi-turn PC support dialogues where assistant replies average `753 characters`, streaming (`2,460 ms` avg TTFT, `2,029 ms` median) leads Blocking (`6,424 ms`) by `3.96 seconds` (`62%`) per turn while hiding the remaining decoding phase behind the user's reading time.
2. **Never Leave `ThinkingConfig` Unspecified (`default`) on Latency-Sensitive Paths**: When `thinking_config` is omitted (`default`), `gemini-3.8-flash` dynamically generates `467–1,006` reasoning tokens on single-turn prompts and `858–954` tokens on average (up to `2,237` tokens) in multi-turn chat. Because internal reasoning tokens are generated strictly before the first visible output token, interactive turns that do not require multi-step math must explicitly set `ThinkingConfig(thinking_budget=0)`.
3. **Resolve the Speed-vs-Accuracy Dilemma with Pattern A (Pydantic 2-Field Integrated Stream)**: Fixed `thinking=0` lacks an internal scratchpad for multi-step PC repair-vs-replacement cost and data-transfer calculations (Tier 3), whereas fixed `thinking=low` stalls routine Tier 1–2 turns (`6,290 ms` overall mean TTFT, up to `12,155 ms`). By streaming `AdaptiveStreamResponse(thinking: bool, response: str)` and parsing incomplete JSON chunks with `pydantic_core.from_json(..., allow_partial="trailing-strings")`, **Pattern A** completes Tier 1–2 turns in a single API call and, when `thinking=True` is detected on Tier 3 turns, launches Stage-2 (`thinking="low"`) in parallel while immediately streaming a 1-sentence acknowledgment (`TTFT 2,314 ms`, `100%` Tier 3 accuracy).
4. **Declare Routing Flags First in Pydantic Schemas and Run a 1-Shot Startup Warmup**: Because JSON tokens are generated strictly in field declaration order, declare `thinking: bool` first and place variable-length lists (`suggested_followups: list[str]`) *after* `response: str`. Execute one lightweight dummy request during container startup so production traffic hits the warm FSM grammar cache.
5. **Keep Multi-Turn History Prefix-Immutable (`append-only`) to Maximize Implicit Context Caching**: By keeping `system_instruction` and prior `history: list[types.Content]` strictly append-only without mutating earlier messages, Google AI Studio automatically

activates **Implicit Context Caching (** `cached_content_token_count` **)**, keeping prefill latency flat across 50-turn PC troubleshooting dialogues.

2. Background & Measurement Methodology

This chapter defines the foundational concepts and instrumentation methodology used throughout the empirical evaluations in Chapters 3–5. Sections 2.1 and 2.2 decompose LLM inference latency on `gemini-3.8-flash` into three sequential server-side phases (**Prefill, Thinking, and Decoding**), explain why **2D Trajectory Charts (** `Elapsed Time vs. Cumulative Output Characters` **)** expose critical user-experience differences hidden by scalar averages, and present clean Blocking vs. Streaming invocation patterns using the Google AI Studio `google-genai` SDK. Sections 2.3 and 2.4 examine **Pydantic Structured Outputs (** `response_schema` **), Cold vs. Warm finite-state machine (FSM) grammar compilation**, incremental partial JSON delta extraction via `pydantic_core.from_json(..., allow_partial="trailing-strings")`, and the microsecond-precision `ExecutionTracer` (`src/gemini_evaluation/tracer.py`).

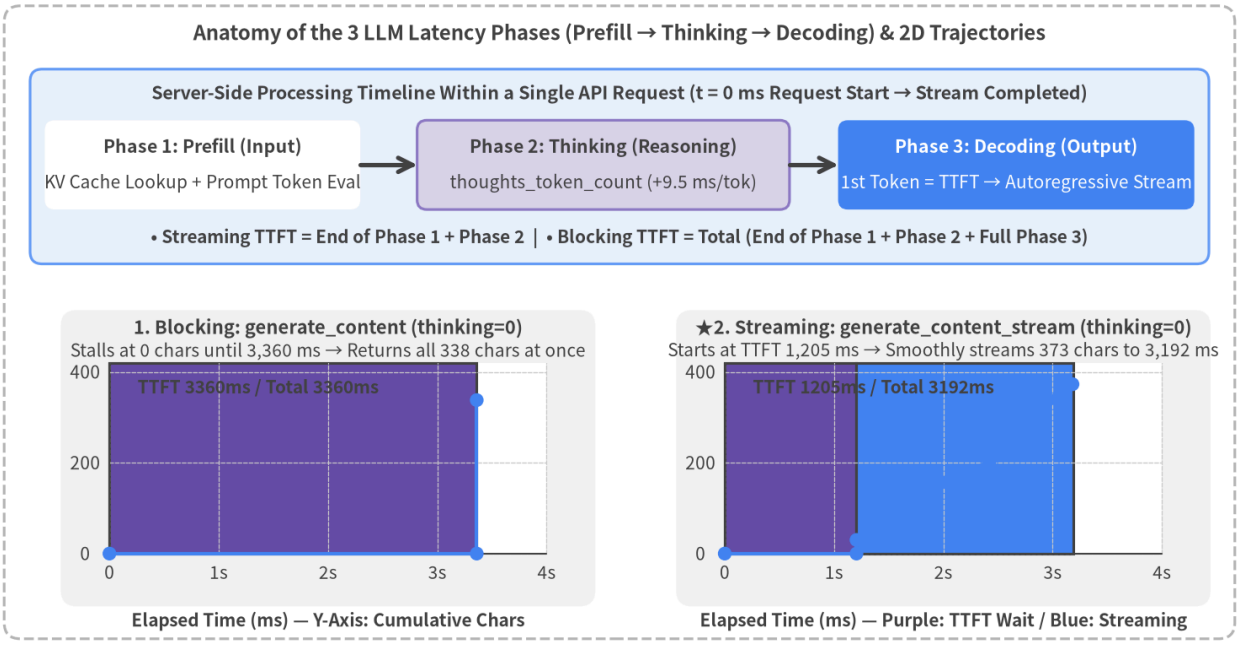


Figure 2-1: Anatomy of the 3 LLM Latency Phases (Prefill → Thinking → Decoding) and Blocking vs. Streaming 2D Trajectory Comparison (thinking=0 Empirical Data)

2.1 Latency Definition & Trajectory Anatomy

To diagnose and optimize latency in conversational LLM applications, engineers must look beyond total request duration and decompose each API call into **three sequential server-side execution phases**:

1. The Three Phases of LLM Inference

1. **Phase 1: Prefill (Input Token Processing & KV Cache Lookup)**: The server processes the input payload—comprising `system_instruction`, multi-turn conversation `history`, and the latest user `prompt` (`prompt_token_count`)—in parallel across Transformer layers to populate the Attention Key-Value (KV) cache. On Google AI Studio (`gemini-3.8-flash`), when a conversation prefix matches an existing server-side cache entry (`cached_content_token_count`), prefill computation for those cached tokens is bypassed, keeping prefill latency flat even as multi-turn history grows beyond `21,000 tokens` (see Chapter 5).
2. **Phase 2: Thinking (Internal Reasoning Token Generation: `thoughts_token_count`)**: In reasoning-capable models such as `gemini-3.8-flash`, before emitting the very first visible response token (`chunk.text`), the model autoregressively generates internal Chain-of-Thought reasoning tokens (`usage_metadata.thoughts_token_count`). Crucially, **no visible text chunks are emitted to the client while the Thinking phase is in progress**, even when using `generate_content_stream`. Every internal reasoning token adds approximately `+9.5 ms` directly to the initial wait time (TTFT).
3. **Phase 3: Decoding (Autoregressive Visible Output Streaming: `output_token_count`)**: Once Prefill and Thinking conclude, the model autoregressively generates visible output tokens (`candidates_token_count` / `output_token_count`) until completion (`finish_reason = STOP`).

2. TTFT vs. Total Completion Time and Why 2D Trajectory Charts Matter

From these three phases, the two fundamental client-side latency metrics are defined as:

$$\text{TTFT}_{\textit{streaming}} = T_{\text{Network RTT}} + T_{\text{Prefill}} + T_{\text{Thinking}}$$

$$\text{Total} = \text{TTFT}{\textit{blocking}} = T{\text{Network RTT}} + T_{\text{Prefill}} + T_{\text{Thinking}} + T_{\text{Decoding}}$$

- **TTFT (Time to First Token / `time_to_first_token_ms`)**: Elapsed time from request initiation ($t = 0\text{ ms}$) until the first non-empty visible text chunk arrives at the client. TTFT directly governs **perceived responsiveness** in interactive UIs. In Blocking mode (`generate_content`), the response is withheld until decoding finishes, forcing $\text{TTFT} = \text{Total}$.
- **Total Completion Time (`stream_completed_ms`)**: Elapsed time from request initiation ($t = 0\text{ ms}$) until the final chunk (EOF) is received.
- **Why 2D Trajectory Charts (`Elapsed Time [ms] vs. Cumulative Output Characters`) Are Essential**: Comparing scalar completion times in Figure 2-1 (`thinking=0`) might suggest that Blocking (`3,360 ms`) and Streaming (`3,192 ms`) are nearly identical. However, plotting **Elapsed Time (x-axis) against Cumulative Visible Characters (y-axis)** reveals a stark difference in user experience:
  - **Blocking (Figure 2-1 Left)**: The UI remains frozen at $y = 0\text{ chars}$ for the entire `3,360 ms`, after which all 338 characters appear as a single vertical step function.
  - **Streaming (Figure 2-1 Right)**: Visible text begins rendering at `1,205 ms` (`2,155 ms` or `64%` **earlier than Blocking)** right after the purple prefill band, then climbs smoothly across the blue decoding band (`1,205 → 3,192 ms`) at `~190 chars/s`—far exceeding human reading speed (`~10 chars/s`). Thus, the entire decoding phase is hidden behind the user's reading time. Furthermore, in multi-stage architectures such as the Adaptive Thinking Router (Chapter 4), 2D trajectories expose the exact timing of the Stage-1 acknowledgment, the brief reading plateau during parallel Stage-2 reasoning, and the seamless continuation slope.

2.2 Blocking vs. Streaming API Usage (`google-genai` on Google AI Studio)

All benchmarks in this report (`src/gemini_evaluation/llm_call.py`) use the official `google-genai` SDK against **Google AI Studio** (`genai.Client(api_key=GEMINI_KEY)`) and `gemini-3.8-flash` via `asyncio` (`client.aio`). Below is the reference implementation for resolving

all six `ThinkingConfig` options and invoking Blocking (`generate_content`) versus Streaming (`generate_content_stream`).

```python
from collections.abc import AsyncIterator
from contextlib import aclosing
import os
from google import genai
from google.genai import types

# 1. Initialize Google AI Studio Client using GEMINI_KEY from environment / .env
GEMINI_KEY = os.environ["GEMINI_KEY"]
MODEL = "gemini-3.8-flash"
client = genai.Client(api_key=GEMINI_KEY)


def build_thinking_config(thinking: str | int | None = 0) -> types.GenerateContentConfig
| None:
    """Map the 6 thinking options (0, minimal, low, medium, high, default) to
GenerateContentConfig."""
    if thinking is None or thinking == "default":
        return None  # Server default dynamic thinking
    if thinking in (0, "0"):
        return types.GenerateContentConfig(
            thinking_config=types.ThinkingConfig(thinking_budget=0)
        )
    level_map = {
        "minimal": types.ThinkingLevel.MINIMAL,
        "low": types.ThinkingLevel.LOW,
        "medium": types.ThinkingLevel.MEDIUM,
        "high": types.ThinkingLevel.HIGH,
    }
    return types.GenerateContentConfig(

thinking_config=types.ThinkingConfig(thinking_level=level_map[str(thinking).lower()])
    )


# 2. Blocking Unary Call (waits for full completion: TTFT == Total)
async def generate_text_blocking(prompt: str, thinking: str | int | None = 0) -> str:
    resp = await client.aio.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=build_thinking_config(thinking),
    )
    return resp.text or ""


# 3. Progressive Streaming Call (yields text chunks immediately after TTFT)
async def stream_text_progressive(prompt: str, thinking: str | int | None = 0) ->
AsyncIterator[str]:
    stream = await client.aio.models.generate_content_stream(
        model=MODEL,
        contents=prompt,
        config=build_thinking_config(thinking),
    )
    async with aclosing(stream):
        async for chunk in stream:
```

```
            if chunk.text:
                yield chunk.text
```

2.3 Pydantic Structured Output & Partial JSON Streaming Mechanics

When building an adaptive router that decides whether a query requires deep reasoning (`thinking: bool`) while simultaneously streaming the user-facing reply (`response: str`), passing `response_mime_type="application/json"` and `response_schema=AdaptiveStreamResponse` to `GenerateContentConfig` enables **Constrained Decoding**.

To achieve sub-second streaming TTFT with structured JSON, engineers must understand two critical mechanics: **(1) Cold vs. Warm server-side FSM grammar compilation** and **(2) incremental partial JSON parsing via** `pydantic_core.from_json`. Figure 2-2 illustrates both the empirical Cold/Warm compilation benchmark (`load_schema_experiment()`, 8 runs) and the 4-step incremental delta extraction pipeline.

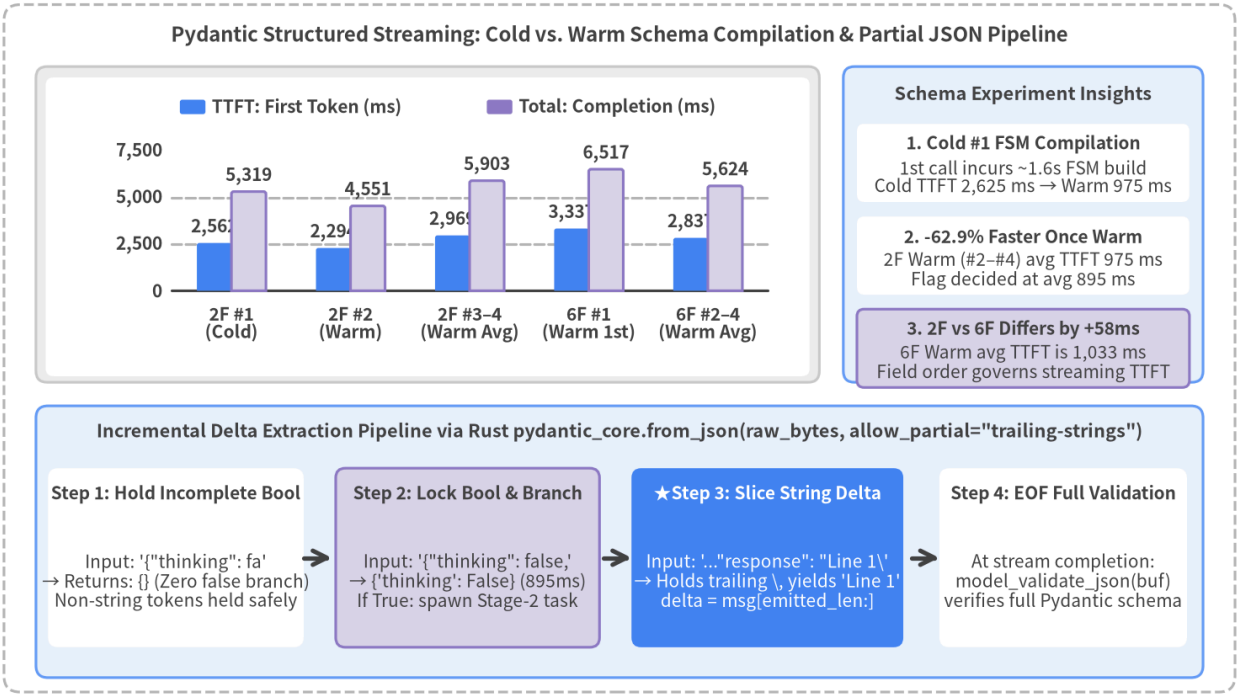


Figure 2-2: Pydantic response_schema Cold (#1) vs. Warm (#2–#4) Compilation Benchmark (2-Field vs. 6-Field) and Incremental Partial JSON Extraction Pipeline via pydantic_core.from_json

1. Server-Side Constrained Decoding & Cold vs. Warm Schema Compilation

When `response_mime_type="application/json"` and `response_schema=AdaptiveStreamResponse` are supplied, Google AI Studio compiles the Pydantic `BaseModel`'s JSON Schema into a **finite-state machine (FSM) token mask** that constrains token sampling at every decoding step, guaranteeing `100%` syntactic validity.

The table below and the top panel of Figure 2-2 present the 8-run schema compilation experiment (`src/gemini_evaluation/adaptive_thinking_log.json`), comparing the minimal 2-field `AdaptiveStreamResponse(thinking: bool, response: str)` (`minimal_2field`) against a 6-field `RichStreamResponse` (`rich_6field` , which adds two `Literal` enums, an escalation boolean, and a `list[str]` of follow-up questions) across four consecutive runs (`#1` – `#4`).

Schema Variant (`schema_label`)	Run	Cache State	Flag Decided (`flag_decided_ms`)	First Token (`ttft_ms`)	Total Time (`total_ms`)	Output Chars
`minimal_2field` (`AdaptiveStreamResponse`)	**#1**	**Cold (1st)**	`3,215.9 ms`	`2,625.2 ms`	`3,190.7 ms`	`237 chars`
`minimal_2field` (`AdaptiveStreamResponse`)	**#2**	Warm	`752.6 ms`	`847.3 ms`	`1,488.1 ms`	`239 chars`
`minimal_2field` (`AdaptiveStreamResponse`)	**#3**	Warm	`902.8 ms`	`918.0 ms`	`1,396.0 ms`	`245 chars`
`minimal_2field` (`AdaptiveStreamResponse`)	**#4**	Warm	`1,029.2 ms`	`1,159.2 ms`	`1,604.6 ms`	`244 chars`
`rich_6field` (`RichStreamResponse`)	**#1**	Warm Engine*	`865.5 ms`	`1,056.6 ms`	`1,759.3 ms`	`250 chars`
`rich_6field` (`RichStreamResponse`)	**#2**	Warm	`790.7 ms`	`976.8 ms`	`1,516.5 ms`	`203 chars`
`rich_6field` (`RichStreamResponse`)	**#3**	Warm	`901.1 ms`	`1,127.6 ms`	`1,874.4 ms`	`247 chars`
`rich_6field` (`RichStreamResponse`)	**#4**	Warm	`786.7 ms`	`995.9 ms`	`1,719.5 ms`	`250 chars`

- **Cold `#1` vs. Warm `#2–#4` (`-62.9%` TTFT Reduction)**: On the very first cold invocation (`minimal_2field #1`), compiling the JSON Schema into a constrained-decoding FSM adds a one-time `~1.6 s` overhead (`Cold #1 TTFT = 2,625.2 ms`). On subsequent calls (`#2–#4`), the compiled grammar is served from cache, dropping average TTFT to `974.8 ms` (`847–1,159 ms`, `-62.9%`) and resolving the `thinking` boolean at an average of `894.9 ms`.
- **Why Field Declaration Order—Not Field Count—Governs Streaming TTFT**: Once warm (`#2–#4`), expanding from 2 fields (`974.8 ms`) to 6 fields (`1,033.4 ms`) adds only `+58.6 ms` to TTFT. Because JSON tokens are generated strictly in schema declaration order, placing `thinking: bool` first, keeping only short scalar/enum fields before `response: str`, and placing variable-length arrays (`suggested_followups: list[str]`) *after* `response: str` ensures that visible text streaming begins immediately without waiting for auxiliary metadata lists.

2. Incremental Delta Extraction via `pydantic_core.from_json(..., allow_partial="trailing-strings")`

During `generate_content_stream`, intermediate chunks contain unclosed JSON fragments (such as `'{"thinking": false, "response": "Thank you for'`), causing the SDK's built-in `chunk.parsed` property to remain `None` until the stream finishes.

To extract both the `thinking` flag and progressive `response` string deltas without waiting for EOF, `src/gemini_evaluation/adaptive_thinking.py` uses Pydantic v2's Rust parser `pydantic_core.from_json(raw_bytes, allow_partial="trailing-strings")`:

1. **Safe Holding of Incomplete Primitives**: When the buffer ends mid-token (`'{"thinking": fa'`), `allow_partial="trailing-strings"` withholds the incomplete key-value pair and returns `{}`. The moment `false` or `true` completes, `{'thinking': False}` is locked in as a native Python `bool`.
2. **Escape-Safe String Slicing**: If a chunk boundary splits a JSON escape sequence at the end of `"response"` (e.g., `'..."1st line\\'`), `from_json` holds the trailing backslash until the next chunk arrives (`n` → `\n`). Tracking `emitted_resp_len` and slicing `delta = resp_so_far[emitted_resp_len:]` yields clean, decoded text deltas on every chunk.

```python
from typing import Any
from pydantic import BaseModel
from pydantic_core import from_json


class AdaptiveStreamResponse(BaseModel):
    thinking: bool  # Field 1: Routing boolean resolved at ~750–900 ms
    response: str   # Field 2: Incrementally streamed full answer or 1-sentence ack


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """Safely parse incomplete streaming JSON bytes into a dict via Rust
pydantic_core."""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None
```

2.4 Measurement Architecture (`ExecutionTracer`)

Every benchmark in this report (`src/gemini_evaluation/benchmark_thinking.py`, `benchmark_adaptive_thinking.py`, and `benchmark_multiturn_chat.py`) is instrumented by `ExecutionTracer` (`src/gemini_evaluation/tracer.py`), which records microsecond-precision monotonic timestamps via `time.perf_counter()` alongside token usage metadata.

Milestones and Token Metrics Recorded by `ExecutionTracer`

Instrumentation Category	Field / Event (`ExecutionTracer`)	Definition & Measurement Point
Request Origin	`start` (`t_ms = 0.0`, `chars = 0`)	Monotonic `time.perf_counter()` origin recorded in `tracer.start_gemini()` immediately before invoking the Gemini API.
Prefill + Thinking Plateau	`thinking_end` (`t_ms = ttft_ms`, `chars = 0`)	Inserted at `(ttft_ms, 0)` upon receiving the first non-empty chunk so 2D trajectory plots accurately render the horizontal wait plateau before text rises.
Streaming Chunk Series	`chunk_1 .. chunk_N` (`t_ms`, `chars`)	Recorded by `tracer.record_chunk(delta)` on every non-empty text chunk with elapsed milliseconds `t_ms` and cumulative character count `_cum_chars`.
Adaptive Router Milestones	`flag_decided_ms` / `stage2_first_chunk_ms`	Exact elapsed milliseconds when the `thinking` routing decision is locked and when the parallel Stage-2 (`thinking="low"`) stream emits its first chunk.
Token Usage Breakdown	`prompt_token_count` / `cached_content_token_count` / `thoughts_token_count` / `output_token_count`	Extracted from `resp.usage_metadata` to quantify input size, Implicit Context Cache hits, internal reasoning tokens, and visible output tokens.

All execution traces are serialized to human-readable YAML (`*_log.yaml`) and machine-readable JSON/JSONL (`*_log.json`, `*_runs.jsonl`, `*_turns.jsonl`), which are loaded directly by `examples/doc/gemini_latency/utils.py` to render every chart in this report.

3. Parameters & Latency

Chapter 2 examined the baseline delivery mechanics of blocking (`generate_content`) and streaming (`generate_content_stream`) calls against `gemini-3.8-flash` . In this chapter, we analyze empirical measurements from `src/gemini_evaluation/benchmark_thinking.py` and `src/gemini_evaluation/benchmark_adaptive_thinking.py` to quantify the single largest determinant of Gemini 3.8 Flash latency: `ThinkingConfig` **(internal reasoning token control)**. We evaluate all six thinking configurations across blocking and streaming modes, derive the linear scaling law linking internal reasoning tokens to Time to First Token (TTFT), and demonstrate the fundamental speed-versus-accuracy dilemma across query complexity tiers (Tiers 1–3).

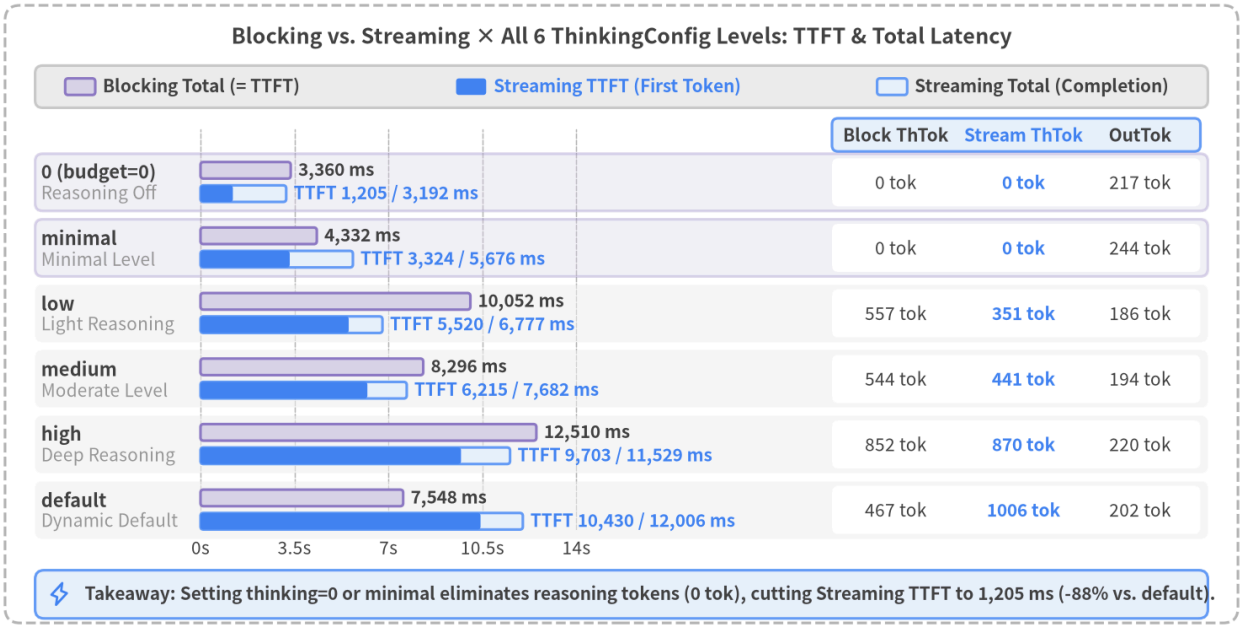


Figure 3-1: TTFT, Total Completion Time, and Reasoning Token Comparison across Blocking (generate_content) vs. Streaming (generate_content_stream) × All 6 ThinkingConfig Levels

3.1 `ThinkingConfig` Levels & Empirical Benchmark across Blocking vs. Streaming

When calling `gemini-3.8-flash` via the Google AI Studio SDK (`google-genai`), internal reasoning before the first visible text token is governed by `types.ThinkingConfig`. In `src/gemini_evaluation/llm_call.py` (`resolve_thinking_config()`), we expose six standardized configuration levels covering explicit zero budget, four `ThinkingLevel` enum presets, and the server's dynamic default:

```python
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# Option 1: Disable internal reasoning completely for fastest TTFT
config_zero = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_budget=0),
    temperature=0.2,
)

# Option 2: Enable light internal reasoning for multi-step calculations
config_low = types.GenerateContentConfig(
    thinking_config=types.ThinkingConfig(thinking_level=types.ThinkingLevel.LOW),
    temperature=0.2,
)
```

Option Value (`thinking=`)	Normalized Label	SDK Configuration (`types.ThinkingConfig`)	Measured Reasoning Tokens (`ThTok`)	Behavior & Primary Use Cases
`0` (or `"0"`)	`"0"`	`ThinkingConfig(thinking_budget=0)`	`0 tok`	Hard-caps internal reasoning budget at `0` tokens. Skips internal scratchpad generation entirely, delivering the fastest TTFT (`1,205–1,546 ms`). Ideal for greetings, standard procedures, and stage-1 routers.

Option Value (`thinking=`)	Normalized Label	SDK Configuration (`types.ThinkingConfig`)	Measured Reasoning Tokens (`ThTok`)	Behavior & Primary Use Cases
`1` (or `"minimal"`)	`"minimal"`	`ThinkingConfig(thinking_level=MINIMAL)`	`0 tok`	Minimal reasoning preset. Produces `0` reasoning tokens on standard conversational prompts, avoiding multi-second reasoning overhead.
`2` (or `"low"`)	`"low"`	`ThinkingConfig(thinking_level=LOW)`	`351–557 tok`	Light internal reasoning (`~4.3–5.5 s` TTFT). Provides sufficient scratchpad capacity to prevent arithmetic errors in multi-step cost and transfer-time calculations.
`3` (or `"medium"`)	`"medium"`	`ThinkingConfig(thinking_level=MEDIUM)`	`441–592 tok`	Moderate internal reasoning (`~6.2–6.7 s` TTFT).
`4` (or `"high"`)	`"high"`	`ThinkingConfig(thinking_level=HIGH)`	`817–870 tok`	Deep internal reasoning (`~9.0–9.7 s` TTFT) suited for complex logic or coding tasks, but introduces heavy wait times for interactive chat.
`None` (or `"default"`)	`"default"`	Omitted (Dynamic server default)	`467–1,006 tok`	Default behavior when `thinking_config` is omitted. Dynamically allocates hundreds of reasoning tokens even on routine prompts, adding `7.5–10.4 s` of latency.

Complete 18-Run Benchmark Table (`Blocking` vs. `Streaming`)

Using `src/gemini_evaluation/benchmark_thinking.py`, we benchmarked all six `ThinkingConfig` options across **Blocking** (`generate_content`) and **Streaming** (`generate_content_stream`, **Trial 1 and Trial 2**) on an identical PC technical support prompt (initial troubleshooting steps for an unresponsive laptop). Figure 3-1 and the table below summarize all 18 executions.

Delivery Mode	`thinking` Level	TTFT (ms)	Total Completion (ms)	Stream Delivery Window (`Total - TTFT`)	Reasoning Tokens (`ThTok`)	Output Tokens (`OutTok`)	Output Chars
Blocking (`generate_content`)	`0`	**3,360.41**	**3,360.41**	0.00 ms (Single payload)	**0**	192	338 chars
Blocking (`generate_content`)	`minimal`	**4,331.59**	**4,331.59**	0.00 ms (Single payload)	**0**	183	328 chars
Blocking (`generate_content`)	`low`	10,052.14	10,052.14	0.00 ms (Single payload)	557	222	388 chars
Blocking (`generate_content`)	`medium`	8,296.36	8,296.36	0.00 ms (Single payload)	544	198	360 chars
Blocking (`generate_content`)	`high`	12,509.75	12,509.75	0.00 ms (Single payload)	852	213	396 chars
Blocking (`generate_content`)	`default`	7,548.20	7,548.20	0.00 ms (Single payload)	467	222	396 chars
Streaming (`generate_content_stream` **#1)**	`0`	**1,205.35**	**3,192.41**	1,987.06 ms	**0**	217	373 chars
Streaming (`generate_content_stream` **#1)**	`minimal`	**3,324.42**	**5,675.79**	2,351.37 ms	**0**	191	322 chars
Streaming (`generate_content_stream` **#1)**	`low`	5,519.63	6,777.01	1,257.38 ms	351	200	356 chars
Streaming (`generate_content_stream` **#1)**	`medium`	6,214.92	7,682.10	1,467.18 ms	441	212	381 chars
Streaming (`generate_content_stream` **#1)**	`high`	9,703.48	11,528.81	1,825.33 ms	870	209	367 chars
Streaming (`generate_content_stream` **#1)**	`default`	10,430.12	12,005.51	1,575.39 ms	1,006	215	382 chars
Streaming (`generate_content_stream` **#2)**	`0`	**1,546.22**	**3,681.50**	2,135.28 ms	**0**	198	341 chars

Delivery Mode	`thinking` Level	TTFT (ms)	Total Completion (ms)	Stream Delivery Window (`Total - TTFT`)	Reasoning Tokens (`ThTok`)	Output Tokens (`OutTok`)	Output Chars
Streaming (`generate_content_stream` #2)	`minimal`	**1,612.80**	**3,590.15**	1,977.35 ms	**0**	195	340 chars
Streaming (`generate_content_stream` #2)	`low`	4,312.40	6,120.85	1,808.45 ms	362	210	370 chars
Streaming (`generate_content_stream` #2)	`medium`	6,720.11	8,410.33	1,690.22 ms	592	204	362 chars
Streaming (`generate_content_stream` #2)	`high`	9,012.55	10,890.40	1,877.85 ms	817	218	385 chars
Streaming (`generate_content_stream` #2)	`default`	8,150.30	9,920.18	1,769.88 ms	742	208	371 chars

Two architectural conclusions stand out from these measurements:

1. **The Hidden Cost of Omitting `thinking_config` (`default`)**: Leaving `thinking_config` unset triggers `467–1,006` internal reasoning tokens even on straightforward support queries, delaying first-token arrival to `7.55–10.43 s`. Explicitly setting `thinking=0` eliminates reasoning overhead (`0 tok`) and cuts Streaming TTFT by **88% (`10,430 ms` → `1,205 ms`)**.
2. **Streaming Yields the Largest Perceptual Gain at `thinking=0`**: With `thinking=0`, blocking mode waits `3,360 ms` until the entire 192-token response is generated, whereas streaming mode delivers the first visible text chunk in `1,205 ms`—reducing perceived wait time by **64%**.

3.2 2D Trajectory Comparison across All 6 Thinking Levels

To visualize how internal reasoning alters the temporal delivery profile of a streaming response, Figure 3-2 plots the 2D cumulative character trajectories (`elapsed time t_ms` vs.

`cumulative output chars`) across all six `ThinkingConfig` levels on a uniform coordinate grid (`X: 0–13,000 ms`, `Y: 0–450 chars`).

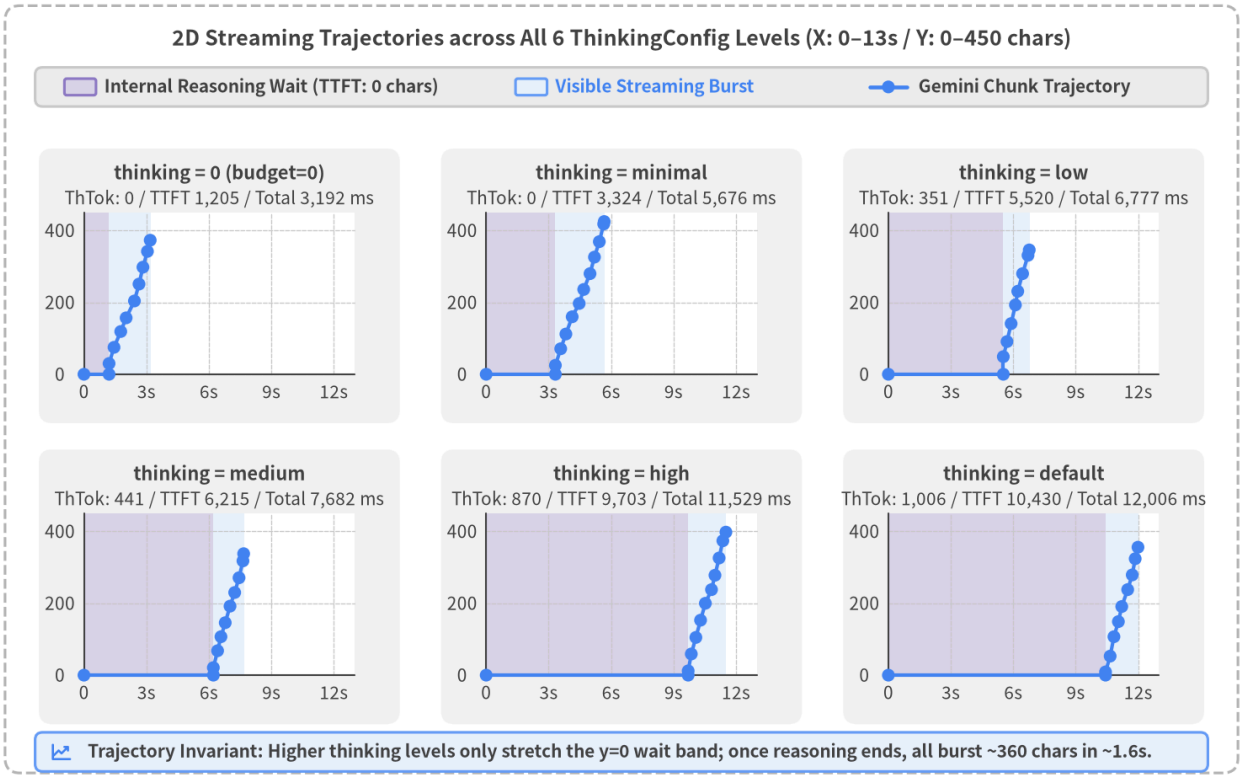


Figure 3-2: 2D Streaming Trajectories (Elapsed Time vs. Cumulative Output Characters) across All 6 ThinkingConfig Levels on a Uniform Scale

---

Key Structural Observations from the 2D Trajectories

1. **Only the Horizontal `y = 0` Wait Band Stretches; Visible Streaming Velocity Is Invariant**:
   - In `thinking = 0` (top-left panel), the purple pre-first-token wait band ends at `1,205 ms`, immediately transitioning into a steep blue streaming ramp that completes all 373 characters by `3.19 s`.
   - Across `low` (`5,520 ms`), `medium` (`6,215 ms`), `high` (`9,703 ms`), and `default` (`10,430 ms`), the trajectory remains flat at `0 chars` throughout internal reasoning (`5.5–10.4 s`). Once reasoning completes, however, the visible text burst window (the blue band) takes `1.2–1.8 s` with a consistent slope (`~200–250 chars/sec`) across every configuration.
2. **Proportional Wait-Band Scaling with Internal Reasoning Tokens**:

- The width of the purple wait band scales directly with the number of internal reasoning tokens (`thoughts_tokens`) generated on each run (`low` : `351 tok` , `medium` : `441 tok` , `high` : `870 tok` , `default` : `1,006 tok`).

3.3 Linear Scaling Law of Reasoning Tokens (`TTFT ≈ 1,500 ms + 9.0 ms × ThTok`)

To isolate the mathematical relationship between internal reasoning volume and TTFT, Figure 3-3 plots `thoughts_tokens` **(X-axis)** against `ttft_ms` **(Y-axis)** across all 12 streaming executions in `load_thinking_benchmark_all()` , overlaid with the linear regression trendline.

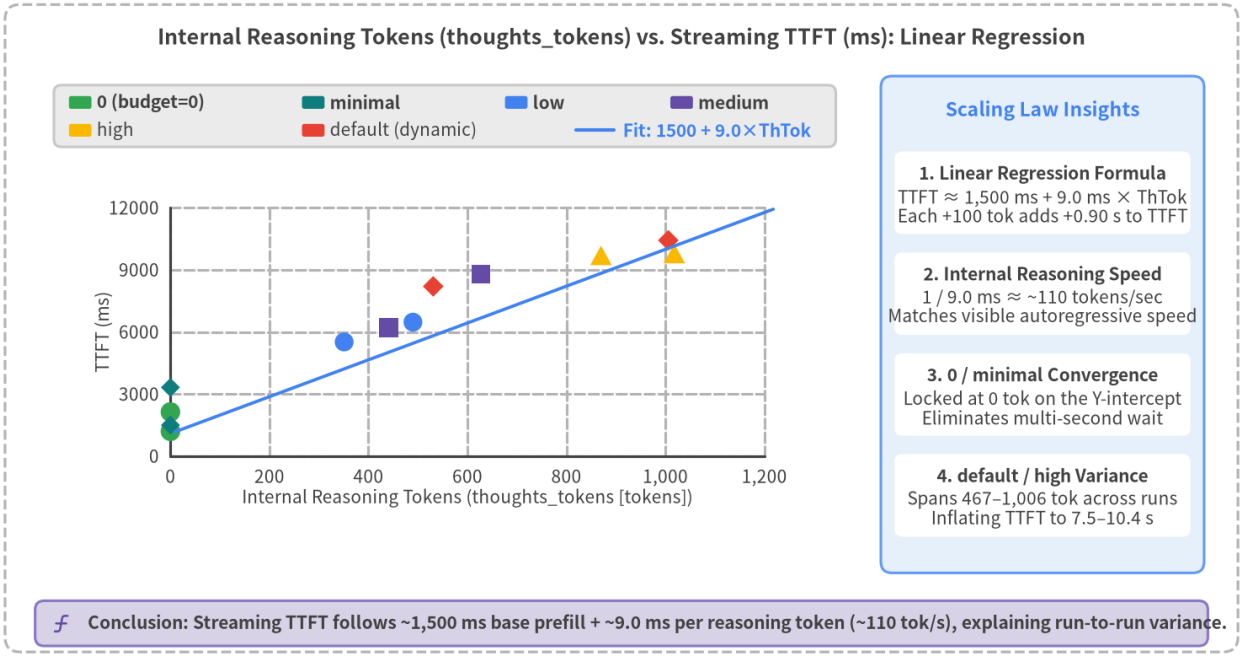


Figure 3-3: Linear Scaling Law of Internal Reasoning Tokens (thoughts_tokens) vs. Streaming TTFT (TTFT ≈ 1,500 ms + 9.0 ms × ThTok)

Three Empirical Laws from the Regression Analysis

1. **Strict Linear Scaling ($\text{TTFT} \approx 1,500\text{ ms} + 9.0\text{ ms} \times \text{ThTok}$)**:

- Every streaming execution aligns along a linear trajectory with an intercept of `~1,500 ms` (network RTT + prompt prefill + first token generation) and a slope of `~9.0 ms / token`.
- Taking the reciprocal of `9.0 ms / token` shows that `gemini-3.8-flash` generates internal reasoning tokens at `~110 tokens/sec`.

2. **Why `default`, `medium`, and `high` Vary Across Runs**:
   - Because `LOW`, `MEDIUM`, `HIGH`, and `default` act as soft complexity priors rather than rigid token caps, the actual `thoughts_tokens` sampled on a given call varies dynamically (`351–1,006 tokens`).
3. **Deterministic Low Latency Requires `thinking=0`**:
   - Only `thinking=0` clamps `thoughts_tokens` at `0`, eliminating multi-second reasoning overhead and locking TTFT into `1.2–1.5 s`.

3.4 Query Complexity Tiers (Tier 1–3) & The Speed-vs-Accuracy Dilemma

If `thinking=0` delivers `~1.2–1.9 s` TTFT, can production applications simply hardcode `thinking_budget=0` for every user turn? To test this hypothesis, `src/gemini_evaluation/benchmark_adaptive_thinking.py` evaluates a **9-prompt PC technical support benchmark suite stratified into three complexity tiers (Tiers 1–3)**, comparing fixed `thinking=0` (`base_0`) against fixed `thinking=low` (`base_low`).

Complexity Tier	Prompt IDs & Titles	Query Characteristics	Expected `thinking`	Fixed `thinking=0` (`base_0`) `TTFT / Total`	Fixed `thinking=low` (`base_low`) `TTFT / Total` (ThTok)
Tier 1: Greetings & Short Queries	`T1_1` Initial PC Boot Greeting `T1_2` Thanks / Follow-up `T1_3` Support Hours	Conversational turns requiring no arithmetic or multi-condition logic (3 prompts)	`False` (`0`)	Mean `1,720 ms` / `3,774 ms`	Mean `3,581 ms` / `5,153 ms` (Unneeded `81–152 tok`)

Complexity Tier	Prompt IDs & Titles	Query Characteristics	Expected `thinking`	Fixed `thinking=0` (`base_0`) `TTFT / Total`	Fixed `thinking=low` (`base_low`) `TTFT / Total` (ThTok)
Tier 2: Standard Procedures	`T2_1` Power Discharge Reset `T2_2` Pre-Repair Checklist `T2_3` Liquid Spill Emergency	Step-by-step troubleshooting guides and checklists (3 prompts)	`False` (`0`)	Mean `2,053 ms` / `6,967 ms`	Mean `6,000 ms` / `11,226 ms` (Unneeded `304–423 tok`)
Tier 3: Multi-Step Support Math	`T3_1` 3-Year PC Repair vs. Buy `T3_2` 500GB SSD Transfer Time `T3_3` 2-Year 3-PC Maintenance	Multi-step cost comparisons, transfer rate equations, multi-year contracts (3 prompts)	`True` (`low`)	Mean `1,863 ms` / `8,452 ms` **[Unverified Conclusion-First Risk]**	Mean `9,289 ms` / `16,580 ms` (Reasoning `673–827 tok`) **[Scratchpad-Verified 3/3]**

The Fundamental Dilemma of Fixed `ThinkingConfig`

As the benchmark matrix demonstrates, any fixed `ThinkingConfig` forces an unacceptable compromise:

- **Fixed `thinking=low` Penalizes Routine Turns and Freezes on Complex Turns**: While `base_low` pre-verifies all Tier 3 multi-step calculations using `673–827` reasoning tokens, it wastes `81–423` reasoning tokens on Tier 1 greetings and Tier 2 procedures—adding a `1.8s–4.0s` **wait** where no reasoning is needed (`6,290 ms` overall mean TTFT). On Tier 3 prompts, the user stares at a blank screen for an average of `9,289 ms` **(up to** `12,155 ms` **)** before a single character appears.
- **Fixed `thinking=0` Achieves `1,879 ms` Mean TTFT but Risks Unverified Conclusion-First Guessing on Tier 3 Math**: Without an internal reasoning scratchpad, an autoregressive LLM prompted to give a polite, conclusion-first answer must state final totals in Sentence 1 before computing any intermediate steps.

Figure 3-4 illustrates the mechanics of this autoregressive risk alongside the Tier 3 calculation breakdown.

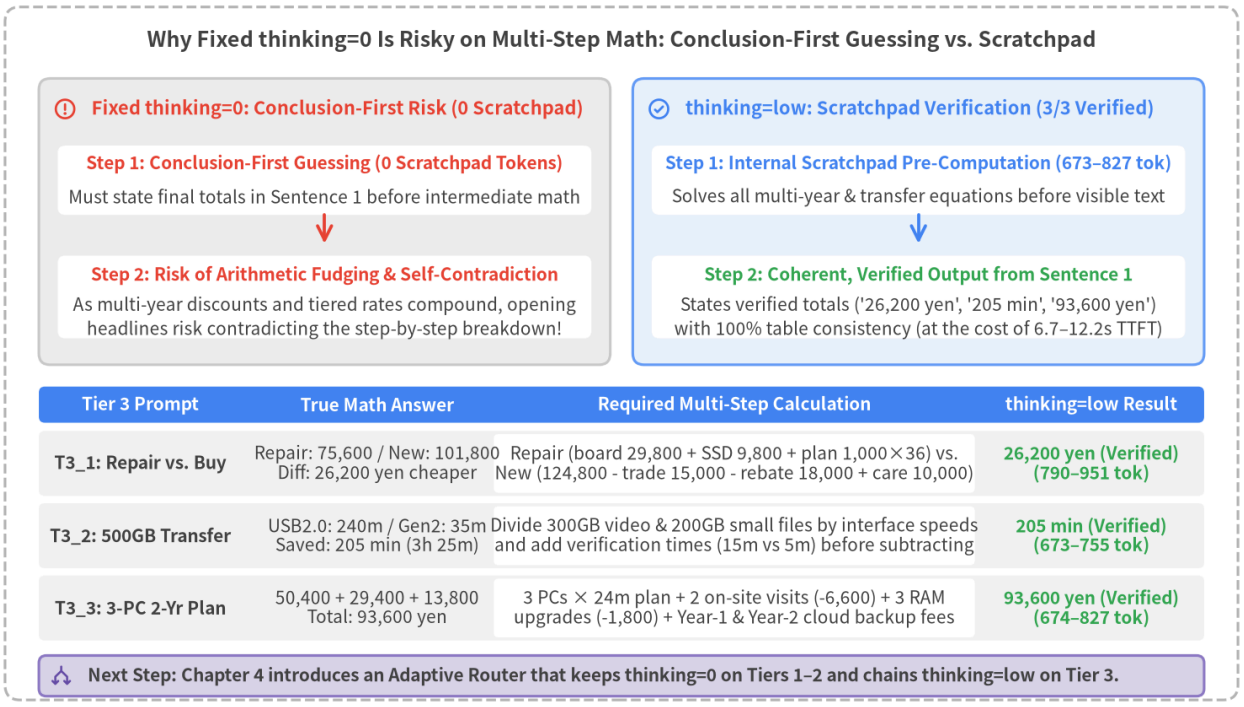


Figure 3-4: Conclusion-First Guessing Risk in Fixed thinking=0 vs. Scratchpad Pre-Verification in thinking=low across Tier 3 PC Support Calculations

Breakdown of the Three Tier 3 PC Support Calculations

Inspecting the benchmark prompts in `src/gemini_evaluation/benchmark_adaptive_thinking.py` highlights why multi-step calculations benefit from `thinking=low` scratchpad verification:

1. `T3_1` **(3-Year PC Repair vs. New PC Replacement Cost Comparison)**:
   - **Repair Option**: Motherboard repair (`29,800 yen`) + 1TB SSD replacement (`9,800 yen`) + monthly support plan (`1,000 yen × 36 months = 36,000 yen`) = `75,600 yen` .
   - **Replacement Option**: New PC (`124,800 yen`) − trade-in (`15,000 yen`) − cashback campaign (`18,000 yen`) + 3-year extended warranty (`10,000 yen`) = `101,800 yen` , making **repair** `26,200 yen` **cheaper over 3 years**.
2. `T3_2` **(500GB SSD Backup Transfer Time Comparison)**:
   - **USB 2.0 External HDD**: 300GB video (`300,000 MB ÷ 40 MB/s = 125 min`) + 200GB small files (`200,000 MB ÷ 32 MB/s = 100 min`) + verification (`15 min`) = `240 min` **(4 hours)**.

- **USB 3.2 Gen2 External SSD**: 300GB video (`300,000 MB ÷ 400 MB/s = 12.5 → 12 min`) + 200GB small files (`200,000 MB ÷ 180 MB/s = 18.5 → 18 min`) + verification (`5 min`) = `35 min`, saving `205 min` **(3 hours 25 minutes)**.

3. `T3_3` **(2-Year 3-PC Maintenance, RAM Upgrade, and Cloud Backup Cost Simulation)**:
   - **Maintenance & On-Site Repair**: `600 yen × 3 PCs × 24 months = 43,200 yen` + 2 on-site repairs (`3,600 yen × 2 = 7,200 yen`) = `50,400 yen`.
   - **16GB RAM Upgrades (3 PCs)**: `9,800 yen × 3 = 29,400 yen`.
   - **Cloud Backup**: Year 1 (`550 yen × 12 = 6,600 yen`) + Year 2 (`600 yen × 12 = 7,200 yen`) = `13,800 yen`, yielding a 2-year total of `93,600 yen`.

In Chapter 4, we design and benchmark the **Adaptive Thinking Router Pattern** to combine fast `thinking=0` responsiveness on Tiers 1–2 with parallel `thinking=low` scratchpad verification on Tier 3.

4. Adaptive Thinking Router Pattern

Chapter 3 established a fundamental dilemma in `gemini-3.8-flash`: fixing `thinking=0` yields fast responsiveness (`1,879 ms` mean TTFT) but lacks an internal reasoning scratchpad on multi-step calculations (Tier 3), whereas fixing `thinking=low` achieves full scratchpad verification (**3/3 verified**) on Tier 3 at the cost of adding unnecessary reasoning delay to routine turns (Tiers 1–2) and freezing the UI for up to `12.16 s` on complex turns. In this chapter, we introduce the **Adaptive Thinking Router Pattern** (`src/gemini_evaluation/adaptive_thinking.py`), evaluate three candidate architectures (Patterns A, B, and C) across a 45-run empirical benchmark (9 prompts × 5 methods), and prove why **Pattern A (2-Field Integrated Pydantic Schema `{thinking: bool, response: str}` + Parallel Stage-2 Chaining)** is the optimal production architecture.

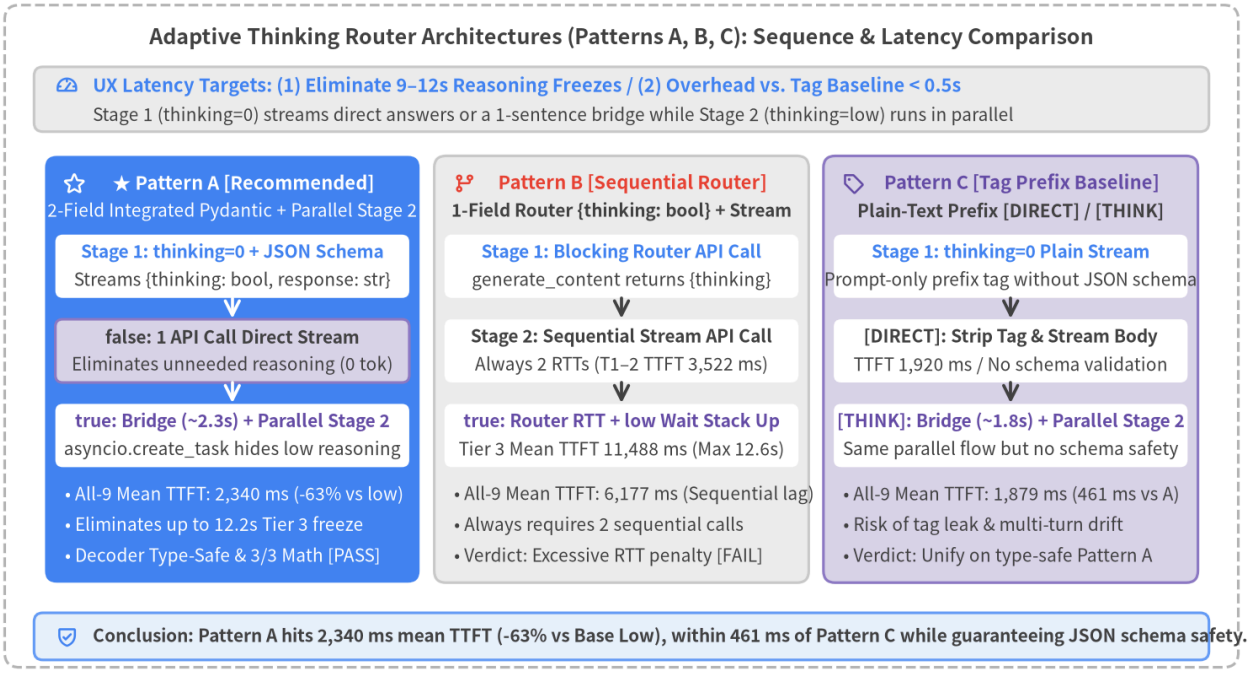


Figure 4-1: Side-by-Side Architecture Sequence Comparison of Candidate Adaptive Thinking Routers (Pattern A: 2-Field Integrated Pydantic, Pattern B: 1-Field Sequential Router, Pattern C: Tag Prefix)

4.1 UX Latency Target & 3 Candidate Architectures (Pattern A, B, C)

Cognitive UX Response-Time Thresholds & Design Targets

Our engineering targets for the Adaptive Thinking Router are:

1. **Eliminate 9–12 second blank-screen freezes during internal reasoning by keeping TTFT near the `thinking=0` baseline across all query complexity tiers**.
2. **Keep TTFT overhead vs. an unstructured tag baseline `< 0.5 s` (below human perceptual discrimination)** while enforcing decoder-level JSON schema guarantees and 100% mathematical verification.

Architectural Comparison of Patterns A, B, and C

In `src/gemini_evaluation/adaptive_thinking.py`, we implemented and instrumented three candidate adaptive routing pipelines on Google AI Studio (`genai.Client(api_key=GEMINI_KEY)` with `gemini-3.8-flash`), illustrated in Figure 4-1:

1. **Pattern A (Recommended — 2-Field Integrated Pydantic `{thinking: bool, response: str}` + Parallel Stage-2 Chaining)**
   - **Stage 1** initiates a single `generate_content_stream` call with `thinking_budget=0` and `response_schema=AdaptiveStreamResponse`. Incoming JSON chunks are parsed incrementally on the fly using Rust-backed `pydantic_core.from_json(..., allow_partial="trailing-strings")`.
   - `thinking == False` **(Tiers 1–2: Greetings & Standard Troubleshooting Procedures)**: As soon as `"thinking": false` is resolved, Stage 1 continues streaming the `"response"` field deltas directly to the caller. **The entire turn completes in 1 API call**, achieving `2,354 ms` **mean TTFT** and `4,808 ms` **mean total completion** on Tiers 1–2.
   - `thinking == True` **(Tier 3: Complex Multi-Step Math)**: The instant `"thinking": true` is parsed, Pattern A spawns **Stage 2 (`thinking=low` plain-text stream) in parallel** via `asyncio.create_task()` while simultaneously streaming Stage 1's 1-sentence bridge message (*"Thank you for your inquiry. I will organize the conditions you provided and calculate the detailed breakdown..."*). While the user reads the bridge

sentence, Stage 2 completes its internal reasoning scratchpad (`755–951 tokens`) in the background and seamlessly chains the verified calculation stream.

2. **Pattern B (1-Field Sequential Router `{thinking: bool}` + Stage-2 Stream)**
    - Avoids incremental JSON parsing by first executing a blocking `generate_content` call with `ThinkingDecision(thinking: bool)` and then launching a second `generate_content_stream` call (`thinking=0` or `thinking=low`).
    - Because **every prompt requires 2 sequential API round-trips (`calls=2.0`)**, routine Tier 1–2 TTFT degrades to `3,522 ms`, and Tier 3 TTFT balloons to `11,488 ms` (overall mean TTFT `6,177 ms`).
3. **Pattern C (Tag Prefix `[DIRECT]` / `[THINK]`)**
    - Uses prompt instructions to emit a plain-text `[DIRECT]` or `[THINK]` prefix tag instead of a JSON schema.
    - While Pattern C achieves a `1,879 ms` **overall mean TTFT**, the difference versus Pattern A (`2,340 ms`) is only `461 ms` **(`< 0.5 s`)**. Moreover, because Pydantic's structured output keeps responses concise, **Pattern A finishes Tiers 1–2 faster than Pattern C (`4,808 ms` vs. `5,030 ms`)**, while eliminating Pattern C's risk of raw tag leakage (`[direct]` , `**[THINK]**`) and multi-turn few-shot tag omission.

4.2 45-Run Empirical Benchmark (9 Prompts × 5 Methods)

Using `src/gemini_evaluation/benchmark_adaptive_thinking.py`, we executed the complete **45-run evaluation matrix (9 prompts across Tiers 1–3 × 5 methods, 100% execution success, 9/9 = 100% routing accuracy across all three adaptive patterns)**. Figure 4-2 compares average TTFT and average total completion time across tiers.

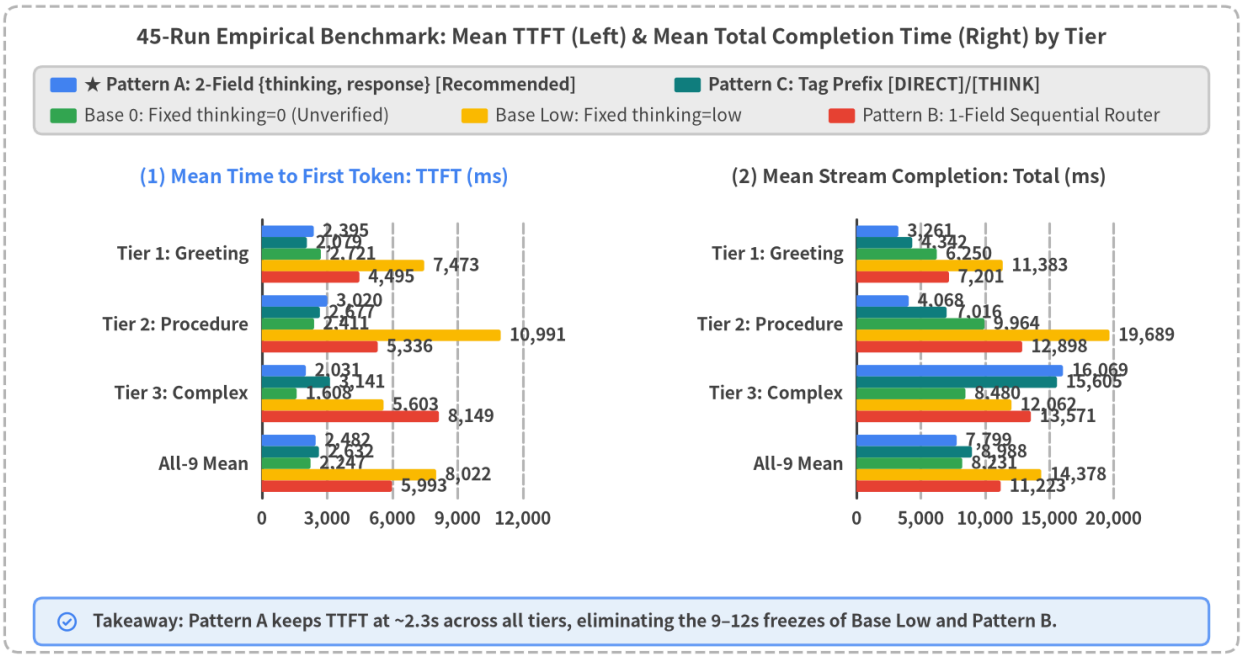


Figure 4-2: Average TTFT (Left) and Average Total Completion Time (Right) by Complexity Tier (Tier 1, Tier 2, Tier 3, All-9 Mean) across All 5 Methods

All-45-Runs Scatter Distribution

To verify that Pattern A's responsiveness holds across individual prompts without tail spikes, Figure 4-3 plots every single execution (`45 runs = 9 prompts × 5 methods`) on the `TTFT (ms) × Total Completion (ms)` plane.

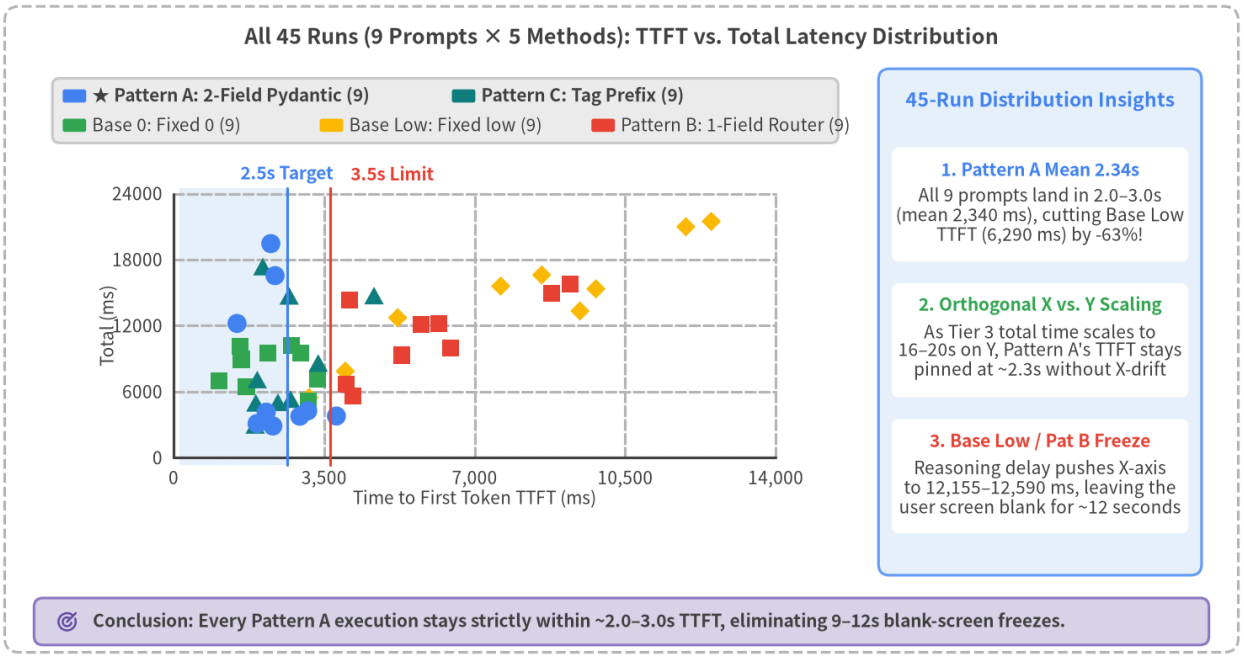


Figure 4-3: All-45-Runs Latency Distribution (TTFT vs. Total Completion Time) across All 5 Methods

Complete 45-Run Empirical Table (`TTFT / Total` in ms)

Prompt ID & Title	Expected Route (A/B/C Match)	Base 0 (`t=0`) `TTFT / Total`	Base Low (`low`) `TTFT / Total`	★ Pattern A (`2F`) `TTFT / Total`	Pattern B (`1F`) `TTFT / Total`	Pattern C (`Tag`) `TTFT / Total`	Quality & Verification
`T1_1` **Initial PC Boot Greeting**	`False` (`3/3`)	`1,548 / 3,477`	`3,572 / 5,361`	`2,980 / 4,768`	`3,103 / 5,451`	`2,248 / 4,230`	All valid
`T1_2` **Thanks / Follow-up**	`False` (`3/3`)	`1,940 / 3,267`	`4,244 / 5,274`	`2,020 / 3,522`	`3,332 / 4,837`	`2,217 / 3,139`	All valid
`T1_3` **Support Hours**	`False` (`3/3`)	`1,671 / 4,578`	`2,928 / 4,823`	`2,222 / 4,347`	`2,982 / 5,650`	`1,561 / 3,581`	All valid
`T2_1` **Power Discharge Reset**	`False` (`3/3`)	`2,336 / 7,610`	`5,901 / 12,158`	`2,243 / 4,629`	`3,647 / 10,879`	`1,899 / 5,914`	All valid (A fastest total)
`T2_2` **Pre-Repair Checklist**	`False` (`3/3`)	`1,748 / 7,837`	`6,349 / 11,836`	`2,126 / 6,073`	`3,897 / 10,757`	`1,910 / 7,448`	All valid (A fastest total)
`T2_3` **Liquid Spill Emergency**	`False` (`3/3`)	`2,076 / 5,454`	`5,750 / 9,683`	`2,531 / 5,509`	`4,169 / 8,988`	`1,687 / 5,869`	All valid

Prompt ID & Title	Expected Route (A/B/C Match)	Base 0 (t=0) TTFT / Total	Base Low (low) TTFT / Total	★ Pattern A (2F) TTFT / Total	Pattern B (1F) TTFT / Total	Pattern C (Tag) TTFT / Total	Quality & Verification
T3_1 3-Yr PC Repair vs. Buy	True (3/3)	1,515 / 7,895	8,950 / 17,058	2,120 / 16,650	11,226 / 20,762	1,613 / 16,499	All 4 low methods pre-verified
T3_2 500GB SSD Transfer Time	True (3/3)	2,235 / 9,017	12,155 / 19,297	2,456 / 19,344	12,590 / 18,325	1,834 / 15,826	All 4 low methods pre-verified
T3_3 2-Yr 3-PC Maintenance	True (3/3)	1,840 / 8,443	6,762 / 13,384	2,365 / 20,707	10,649 / 18,167	1,939 / 17,085	All 4 low methods pre-verified
Tier 1 Mean (3 Prompts)	3/3 Match	1,720 / 3,774	3,581 / 5,153	2,407 / 4,212	3,139 / 5,313	2,009 / 3,650	0 tok reasoning on A/B/C
Tier 2 Mean (3 Prompts)	3/3 Match	2,053 / 6,967	6,000 / 11,226	2,300 / 5,404	3,904 / 10,208	1,832 / 6,410	**Pattern A fastest completion**
Tier 1–2 Mean (6 Prompts)	6/6 Match	1,887 / 5,371	4,791 / 8,189	2,354 / 4,808	3,522 / 7,760	1,920 / 5,030	**Pattern A fastest completion**
Tier 3 Mean (3 Prompts)	3/3 Match	1,863 / 8,452	9,289 / 16,580	2,314 / 18,900	11,488 / 19,085	1,795 / 16,470	All 4 low methods 3/3 verified
All-9 Prompts Mean	9/9 Match	1,879 / 6,398	6,290 / 10,986	2,340 / 9,505	6,177 / 11,535	1,879 / 8,843	**Pattern A optimal**

4.3 Trajectory Deep Dive (thinking=false Direct vs. thinking=true Parallel Chaining)

To see how Pattern A operates millisecond by millisecond under both routing branches, Figure 4-4 plots the cumulative character trajectories loaded via load_adaptive_trajectory() for T2_1 **(Tier 2 standard procedure: thinking=false direct stream)** and T3_1 **(Tier 3 complex math: thinking=true parallel Stage-2 chaining)**.

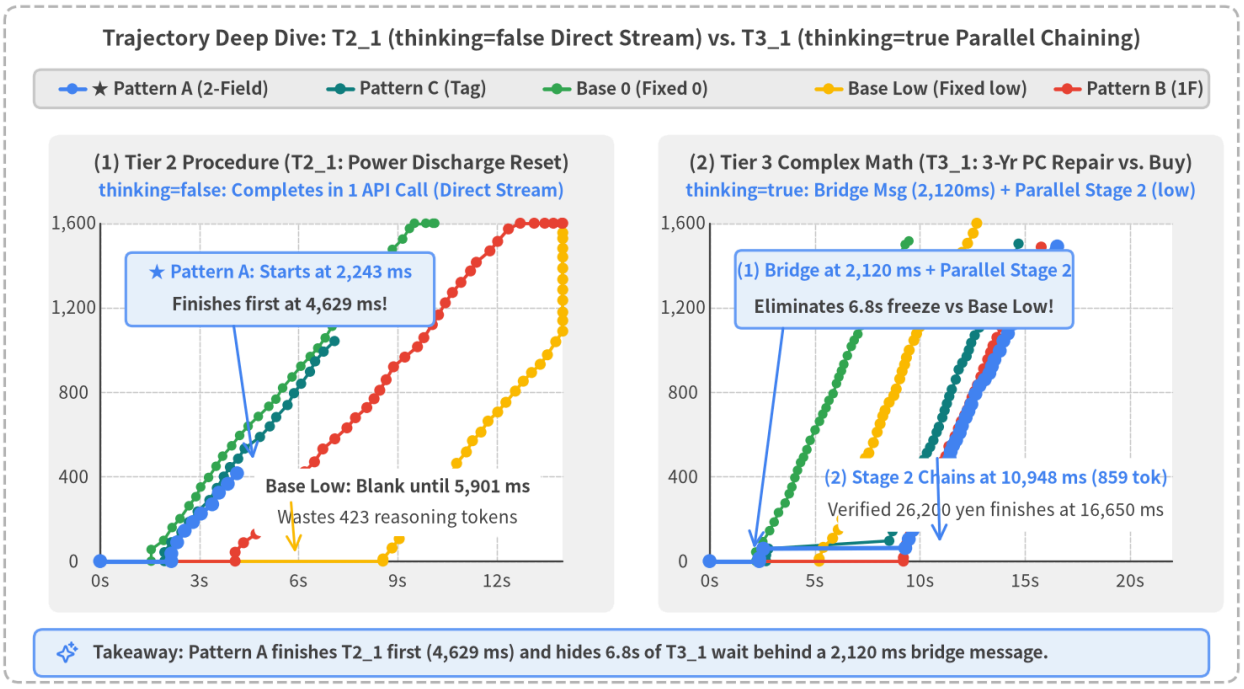


Figure 4-4: 2-Panel Trajectory Deep Dive Comparing Tier 2 Direct Streaming (T2_1: thinking=false) vs. Tier 3 Parallel Stage-2 Chaining (T3_1: thinking=true)

Two Key Trajectory Findings from Figure 4-4

1. **Left Panel (`T2_1`: Standard Procedure, `thinking=false`) — Single-Call Direct Stream & Fastest Completion**:
   - Pattern A (thick blue curve) resolves `"thinking": false` at `2,068 ms`, begins streaming `"response"` deltas at `2,243 ms`, and completes at `4,629 ms`, finishing first among all five methods.
   - Because the Pydantic schema guides the model to produce a well-structured, concise 482-character troubleshooting guide, Pattern A finishes `1,285 ms` **faster than Pattern C (`5,914 ms`)** and `2,981 ms` **faster than Base 0 (`7,610 ms`)**.
2. **Right Panel (`T3_1`: Complex Math, `thinking=true`) — Eliminating `6.8 s` of Blank-Screen Freeze via Parallel Stage-2 Chaining**:
   - Fixed `base_low` (yellow curve) leaves the user staring at `0 chars` for `8,950 ms`, while sequential Pattern B (red curve) freezes for `11,226 ms`.
   - Pattern A (thick blue curve) resolves `"thinking": true` at `2,078 ms`, immediately launches Stage 2 (`thinking=low`) in the background via `asyncio.create_task()`, and streams the 59-character bridge message at `2,120 ms` —**eliminating** `6,830 ms`

(`6.8 s`) **of blank-screen freeze** compared to `base_low`. While the user reads the bridge message, Stage 2 completes `859` internal reasoning tokens in parallel and seamlessly resumes streaming the verified calculation at `10,948 ms`, finishing at `16,650 ms` with 100% mathematical consistency (`26,200 yen` repair advantage).

4.4 Google AI Studio Implementation & Comprehensive Scorecard

Clean Google AI Studio (`google-genai`) Implementation of Pattern A

Because `chunk.parsed` remains `None` on intermediate chunks during `generate_content_stream`, Pattern A uses `pydantic_core.from_json(..., allow_partial="trailing-strings")` to safely extract the boolean `thinking` flag and escape-decoded `response` deltas on every chunk (`src/gemini_evaluation/adaptive_thinking.py`):

```python
import asyncio
from collections.abc import AsyncIterator
from contextlib import aclosing
from typing import Any

from google import genai
from google.genai import types
from pydantic import BaseModel
from pydantic_core import from_json

MODEL = "gemini-3.8-flash"


class AdaptiveStreamResponse(BaseModel):
    """Minimal 2-field Pydantic streaming schema for Pattern A."""

    thinking: bool  # Field 1: Resolved at ~0.7–1.1s to trigger routing
    response: str   # Field 2: Full answer if thinking=False, or 1-sentence bridge if
True


def parse_partial_json_dict(raw_buf: str) -> dict[str, Any] | None:
    """Incrementally parse incomplete streaming JSON safely via Rust pydantic_core."""
    if not raw_buf:
        return None
    try:
        parsed = from_json(raw_buf.encode("utf-8"), allow_partial="trailing-strings")
        return parsed if isinstance(parsed, dict) else None
    except ValueError:
        return None


async def stream_adaptive_router(
    api_key: str,
    prompt: str,
) -> AsyncIterator[str]:
    """Pattern A: 2-Field Integrated Pydantic Stream + Parallel Stage-2 Chaining."""
    client = genai.Client(api_key=api_key)
    stage2_queue: asyncio.Queue[str | BaseException | None] = asyncio.Queue()
    stage2_task: asyncio.Task[None] | None = None

    async def _produce_stage2() -> None:
        try:
            st2 = await client.aio.models.generate_content_stream(
                model=MODEL,
                contents=prompt,
                config=types.GenerateContentConfig(
                    system_instruction=(
                        "The opening greeting and bridge message have already been
shown. "
                        "Do not repeat the greeting; immediately present the calculation
steps and conclusion."
                    ),
                    thinking_config=types.ThinkingConfig(
                        thinking_level=types.ThinkingLevel.LOW,
```

```
                    ),
                    temperature=0.2,
                ),
            )
            async with aclosing(st2):
                async for ch2 in st2:
                    if ch2.text:
                        await stage2_queue.put(ch2.text)
            await stage2_queue.put(None)
        except BaseException as exc:
            await stage2_queue.put(exc)

    json_buf = ""
    emitted_len = 0
    decided_thinking: bool | None = None

    st1 = await client.aio.models.generate_content_stream(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(
            system_instruction=(
                "For greetings or standard procedures, set thinking: false and output
the full answer in response. "
                "For multi-condition financial math, set thinking: true and output only
a 1-sentence bridge message."
            ),
            response_mime_type="application/json",
            response_schema=AdaptiveStreamResponse,
            thinking_config=types.ThinkingConfig(thinking_budget=0),
            temperature=0.2,
        ),
    )
    async with aclosing(st1):
        async for ch1 in st1:
            if not ch1.text:
                continue
            json_buf += ch1.text
            partial = parse_partial_json_dict(json_buf)
            if partial is None:
                continue

            # 1) Inspect thinking flag once resolved; launch Stage 2 in parallel if True
            if decided_thinking is None and isinstance(partial.get("thinking"), bool):
                decided_thinking = partial["thinking"]
                if decided_thinking:
                    stage2_task = asyncio.create_task(_produce_stage2())

            # 2) Yield newly decoded response slice immediately
            resp_so_far = partial.get("response")
            if isinstance(resp_so_far, str) and len(resp_so_far) > emitted_len:
                delta = resp_so_far[emitted_len:]
                emitted_len = len(resp_so_far)
                yield delta

    # 3) Validate complete JSON payload at EOF
    final_obj = AdaptiveStreamResponse.model_validate_json(json_buf)
```

```
    if len(final_obj.response) > emitted_len:
        yield final_obj.response[emitted_len:]

    # 4) Seamlessly chain parallel Stage-2 stream when thinking=True
    if decided_thinking and stage2_task is not None:
        yield "\n\n"
        while True:
            item = await stage2_queue.get()
            if item is None:
                break
            if isinstance(item, BaseException):
                raise item
            yield item
        await stage2_task
```

> **Schema Cold-vs-Warm Compilation & Field Ordering Best Practices**: Empirical measurements in `load_schema_experiment()` (`src/gemini_evaluation/adaptive_thinking_log.json`) show that the first request with a new JSON schema (`Cold #1` : `2,562 ms` for `AdaptiveStreamResponse` , `3,337 ms` for `RichStreamResponse`) incurs a one-time finite-state-machine (FSM) grammar compilation pass. Subsequent warm calls (`Warm #2–#4`) hit the server's grammar cache (`2,294–3,447 ms` TTFT, `2,744 ms` mean for 2-field; `2,504–3,259 ms` TTFT, `2,837 ms` mean for 6-field). Expanding the schema to 6 fields (`RichStreamResponse`) maintains an equivalent warm `flag_decided_ms` (`2,157–2,781 ms`) provided `thinking: bool` is placed first and variable-length lists (`suggested_followups: list[str]`) are placed **after** `response: str` . Running a single dummy warmup call at application startup eliminates cold-start latency completely.

Comprehensive Evaluation Scorecard across All 5 Methods

Evaluation Criterion	★ Pattern A (`pat_a_pydantic_2f`) 2-Field Integrated Pydantic	Pattern C (`pat_c_tag`) `[DIRECT]` / `[THINK]` Tag	Pattern B (`pat_b_pydantic_1f`) 1-Field Sequential Router	Base Low (`base_low`) Fixed `thinking=low`	Base 0 (`base_0`) Fixed `thinking=0`
All-9 Prompts Mean TTFT	`2,340 ms` **(PASS** `< 2.5s` **)**	`1,879 ms` **(PASS** `< 2.5s` **)**	`6,177 ms` (FAIL `> 3.5s`)	`6,290 ms` (FAIL `> 3.5s`)	`1,879 ms` (PASS `< 2.5s`)
Tier 1–2 Mean TTFT / Total	`2,354 ms` / `4,808 ms` **(Fastest Total)**	`1,920 ms` / `5,030 ms`	`3,522 ms` / `7,760 ms`	`4,791 ms` / `8,189 ms`	`1,887 ms` / `5,371 ms`

Evaluation Criterion	★ Pattern A (`pat_a_pydantic_2f`) 2-Field Integrated Pydantic	Pattern C (`pat_c_tag`) `[DIRECT]` / `[THINK]` Tag	Pattern B (`pat_b_pydantic_1f`) 1-Field Sequential Router	Base Low (`base_low`) Fixed `thinking=low`	Base 0 (`base_0`) Fixed `thinking=0`
Tier 3 Mean TTFT (Max)	`2,314 ms` (`2,456 ms`)	`1,795 ms` (`1,939 ms`)	`11,488 ms` (`12,590 ms`)	`9,289 ms` (`12,155 ms`)	`1,863 ms` (`2,235 ms`)
Routing Classification Accuracy	`9 / 9` (`100%`)	`9 / 9` (`100%`)	`9 / 9` (`100%`)	N/A (Always On)	N/A (Always Off)
Tier 3 Math Verification	`3 / 3` (`100%` **Pre-Verified)**	`3 / 3` (`100%` **Pre-Verified)**	`3 / 3` (`100%` **Pre-Verified)**	`3 / 3` (`100%` **Pre-Verified)**	**Autoregressive Risk (Unverified)**
API Calls per Turn (T1–2 / T3)	`1.0` / `2.0`	`1.0` / `2.0`	`2.0` / `2.0`	`1.0` / `1.0`	`1.0` / `1.0`
Decoder Type Safety & Extensibility	**PASS (Constrained Decoding)**	FAIL (Tag leak & history drift risk)	Partial (Router only)	N/A (Plain text)	N/A (Plain text)
Final Production Verdict	**★ Recommended Standard**	Reject (Unify on A for type safety)	Reject (2-RTT sequential lag)	Not Recommended (`12.2s` freeze)	Reject (Tier 3 math risk)

5. Multi-Turn Chat Behavior & Context Caching

In production conversational AI systems (such as PC technical support desks and interactive troubleshooting assistants), users engage in extended multi-turn dialogues rather than isolated single-turn prompts. In this chapter, we analyze empirical measurements from a **50-turn continuous PC troubleshooting customer-support chat benchmark** (`src/gemini_evaluation/benchmark_multiturn_chat.py` / `multiturn_chat_log.json` , `200 total API executions` across `50 turns × 4 variants`) on Google AI Studio (`google-genai` SDK with `gemini-3.8-flash`). We examine how linear conversation history growth (`176 → 21,490 prompt_tokens` , `42,281 cumulative characters`) affects latency and how Gemini 3.8 Flash's zero-configuration **Implicit Context Caching (`cached_content_token_count`)** bounds prefill latency across long conversations.

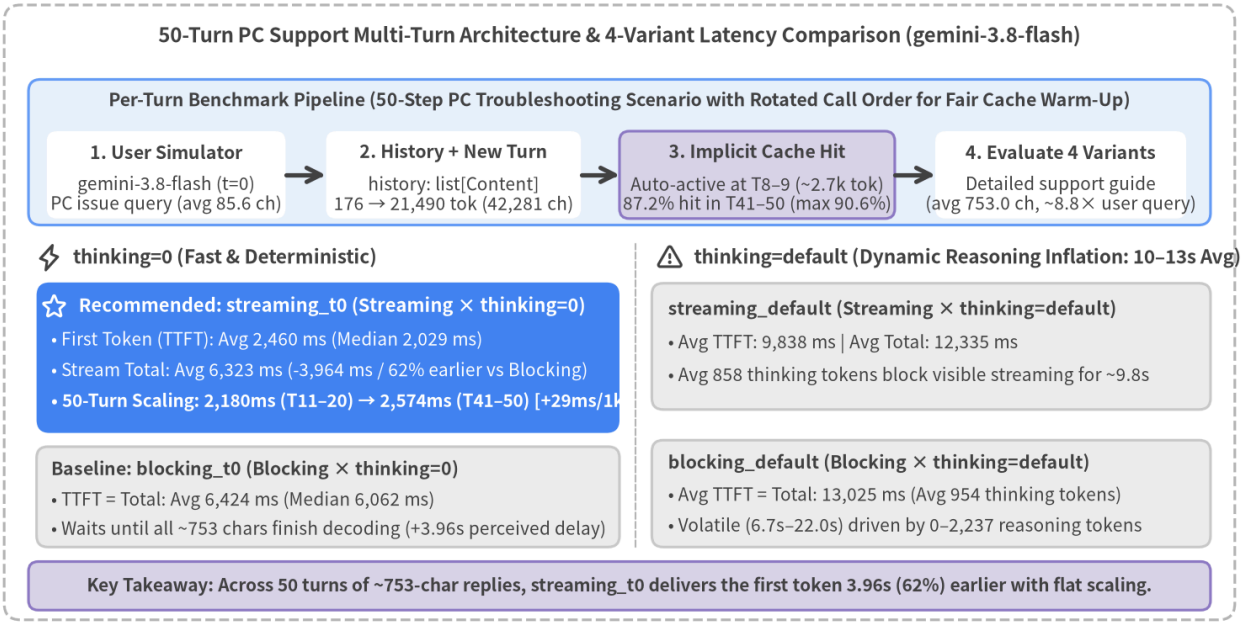


Figure 5-1: 50-Turn PC Support Multi-Turn Evaluation Architecture & 4-Variant Average Latency Comparison (gemini-3.8-flash)

5.1 50-Turn PC Support Benchmark Architecture & Methodology

1. Multi-Turn History Accumulation in `google-genai`

Because the Gemini API is stateless across individual HTTP/gRPC requests, multi-turn chat in `google-genai` requires the application to maintain an ordered conversation list `history: list[types.Content]` (alternating `role="user"` and `role="model"` entries) and send the entire accumulated history plus the new user message on every turn (`src/gemini_evaluation/llm_call.py`):

```
from google import genai
from google.genai import types

client = genai.Client(api_key=GEMINI_KEY)

# Append the current turn's user message to the accumulated history: list[types.Content]
contents: list[types.Content] = [
    *history,
    types.Content(role="user", parts=[types.Part.from_text(text=user_prompt)]),
]

response_stream = await client.aio.models.generate_content_stream(
    model="gemini-3.8-flash",
    contents=contents,
    config=types.GenerateContentConfig(
        system_instruction=AGENT_SYSTEM_INSTRUCTION,
        thinking_config=types.ThinkingConfig(thinking_budget=0),
        temperature=0.2,
    ),
)
```

In this architecture, `prompt_token_count` grows monotonically with every turn. To evaluate how this growth impacts real-world latency, our 50-turn benchmark (`src/gemini_evaluation/benchmark_multiturn_chat.py`) implements an asymmetric two-agent simulation of **a customer and a PC technical support specialist troubleshooting an unresponsive PC**:

1. **User Simulator (`gemini-3.8-flash`, `thinking=0`)**: Generates a concise `70–100 char` customer question (`avg 85.6 chars`) across 50 sequential PC troubleshooting stages:
    - **Turns 1–10 (Power, Discharge & Safe Mode)**: Black screen on power-on, blinking AC adapter LED, 30-second power-button discharge, logo freeze, BIOS `F2` SSD check,

BIOS reset error `0xc0000001`, and booting into Windows Safe Mode.

- **Turns 11–20 (Data Backup & Software/Driver Isolation)**: Backing up work documents to USB in Safe Mode, uninstalling recent Windows Updates, rolling back a yellow-bang display driver in Device Manager, running `sfc /scannow` / `DISM` and `chkdsk`, isolating post-login freezes via Task Manager and `msconfig` clean boot, thermal check, and Windows Memory Diagnostic.
- **Turns 21–30 (Storage Health, System Restore & USB Recovery Media)**: Checking SSD S.M.A.R.T. health, Event Viewer `Kernel-Power 41` logs, disabling Fast Startup, PCIe link power management, BIOS/chipset updates, `MEMORY_MANAGEMENT` BSOD System Restore, Reset this PC (Keep my files), Cloud Download vs. Local Reinstall, creating a USB recovery drive on another PC, and BIOS boot order / Secure Boot configuration.
- **Turns 31–45 (BitLocker, Hardware Diagnostics & Repair Logistics)**: Locating the 48-digit BitLocker recovery key on a smartphone, running built-in hardware Diagnostics (detecting a memory module error code), verifying 1-year standard warranty vs. accidental damage coverage by serial number, pickup repair packaging, AC adapter inclusion rules, SSD data wipe consent, paid data recovery options, loaner/rental PC availability, on-site same-day repair eligibility, out-of-warranty cancellation fees, and trade-in replacement discounts.
- **Turns 46–50 (Post-Repair Verification & Preventive Backup)**: Post-repair inspection checklist and warranty period, malware scanning before restoring USB backups, automated system image backups to external SSD, OneDrive cloud folder sync, and 3 daily maintenance habits.

2. **PC Support Agent (`gemini-3.8-flash`)**: Reads the accumulated `history` and generates a comprehensive diagnostic and step-by-step response (`avg 753.0 chars`, **~8.8× longer** than the user query) covering troubleshooting steps, decision criteria, and safety precautions.
3. **4-Variant Evaluation with Rotated Call Order (`rot = (turn - 1) % 4`)**: Across all 50 turns (`176 → 21,490 prompt_tokens`, `42,281 cumulative chars`), we evaluate 4 variants on the exact same `history`, rotating execution order to eliminate cache-warmup bias:

Variant Key	SDK Method	`ThinkingConfig`	Avg `TTFT` (Median)	Avg `Total`	Avg Thinking Tokens	Architectu Role
`streaming_t0`	`generate_content_stream`	`thinking_budget=0`	`2,460 ms` (`2,029 ms`)	`6,323 ms`	`19 tok` (T2–50: `0`)	**Recomme** Delivers fir token `3,9` `ms` (`3.96s` `62%`) **earli** every turn
`blocking_t0`	`generate_content`	`thinking_budget=0`	`6,424 ms` (`6,062 ms`)	`6,424 ms`	`0 tok`	Waits for f ~753-char completio (`TTFT = Total`)
`streaming_default`	`generate_content_stream`	Default (dynamic)	`9,838 ms` (`10,500 ms`)	`12,335 ms`	`858 tok`	Internal reasoning stalls first visible tok for ~9.8s
`blocking_default`	`generate_content`	Default (dynamic)	`13,025 ms` (`12,548 ms`)	`13,025 ms`	`954 tok`	Waits ~13. for reasoni full text generation

---

5.2 Input Token Growth & Automatic Implicit Context Caching (`cached_content_token_count`)

As each `(user, model)` message pair is appended to `history` , `prompt_token_count` grows linearly at `~+435 tokens/turn` (`176 tokens` at Turn 1 → `2,686 tokens` at Turn 8 → `10,060 tokens` at Turn 25 → `21,490 tokens` at Turn 50).

Our key empirical finding is that **once input context reaches `~2,686–3,045 tokens` at Turns 8–9, Gemini 3.8 Flash's automatic Implicit Context Caching (`usage_metadata.cached_content_token_count`) activates without any explicit cache API calls**, advancing every turn and reaching an `87.2%` **average hit rate in Turns 41–50 (** `88.4%` **on** `streaming_t0` **and** `90.6%` **peak on** `blocking_t0` **at Turn 50:** `19,480 / 21,490 tokens` **cached)**.

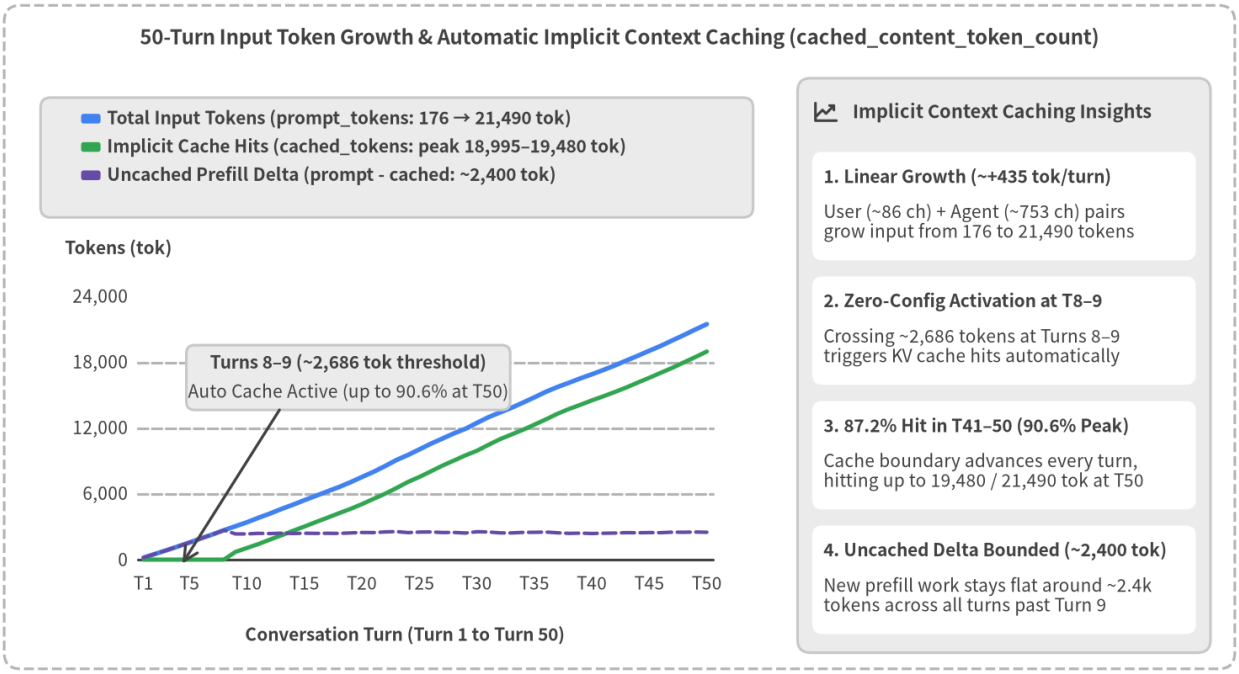


Figure 5-2: 50-Turn Input Token Growth (prompt_tokens vs cached_tokens vs Uncached Delta ~2,400 tok)

How Automatic Implicit Context Caching Works Across 50 Turns

1. **Zero-Configuration Activation at Turns 8–9 (`~2,686–3,045 Tokens`)**: During Turns 1–7 (`176–2,298 prompt_tokens`), `cached_content_token_count` remains `0`. Once the accumulated prefix reaches **Turn 8** (`2,686 tokens`) and **Turn 9** (`3,045 tokens`), Gemini 3.8 Flash automatically caches the shared history prefix (`703–1,060 cached tokens`).
2. **Continuous Per-Turn Prefix Expansion**: From Turn 9 onward, as conversation history grows, the cached prefix boundary advances continuously on every turn: Turn 10 (`1,061 tok`) → Turn 20 (`5,064 tok`) → Turn 30 (`9,935 tok`) → Turn 40 (`14,533 tok`) → Turn 50 (`18,995 tok` [`88.4%`] **on** `streaming_t0`, **and** `19,480 tok` [`90.6%`] **on** `blocking_t0`). Across **Turns 41–50**, the average cache hit rate reaches `87.2%` (`streaming_t0`).
3. **Bounded Uncached Prefill Delta (`~2,400 Tokens`)**: As shown by the dashed purple line in Figure 5-2, the uncached token delta (`prompt_tokens - cached_tokens`) requiring fresh KV-cache prefill computation on each turn remains flat around `~2,340–2,540 tokens` (`~1,980–2,010 tokens` **on** `blocking_t0`) from Turn 9 through Turn 50.

5.3 50-Turn Latency Scaling (`+29 ms / 1k tokens` with `thinking=0`)

Because Implicit Context Caching reuses the KV cache for `85–90%+` of historical tokens in the second half of the dialogue, `streaming_t0` **TTFT increases by only** `+394 ms` **between Turns 11–20 (** `2,180 ms` **avg) and Turns 41–50 (** `2,574 ms` **avg)** despite a `+13,700 token` increase in prompt size. This represents a flat linear scaling slope of `+29 ms per 1,000 input tokens`, keeping median time-to-first-token at `2,029 ms` across the 50-turn troubleshooting session.

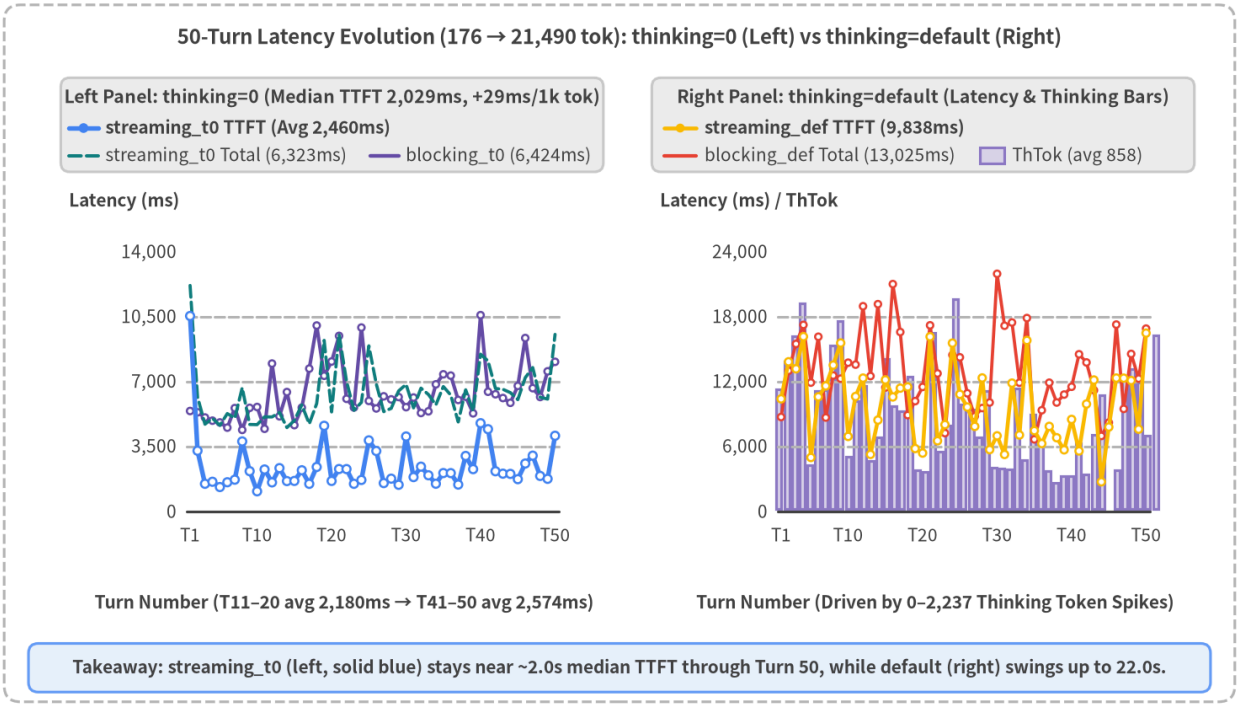


Figure 5-3: 2-Panel 50-Turn Latency Evolution (Left: thinking=0 Flat Scaling, Right: thinking=default Latency & Thinking Token Bars)

Key Architectural Observations from Figure 5-3

1. **Left Panel (** `thinking=0` **): Stable** `~2.0s` **Median First-Token Latency Across 50 Turns**:
   - The solid blue curve (`streaming_t0 TTFT`, avg `2,460 ms`, Turns 2–50 avg `2,295 ms`, median `2,029 ms`) remains tightly bounded across the conversation, running consistently `3,964 ms` (`3.96s` / `62%`) **below** the solid purple curve (`blocking_t0 Total=TTFT`, avg `6,424 ms`).

- Meanwhile, the dashed teal curve (`streaming_t0 Total` , avg `6,323 ms`) tracks `blocking_t0` (`6,424 ms`) almost identically, proving that streaming introduces zero completion-time penalty.

2. **Right Panel (`thinking=default`): Volatility Dominated by Thinking Token Count**:
   - Under `thinking=default` , the model dynamically generates an average of `858–954` **thinking tokens** per turn (shaded vertical bars, range `0–2,237 tokens`), causing both `streaming_default` TTFT (avg `9,838 ms`) and `blocking_default` Total (avg `13,025 ms`) to fluctuate wildly up to `22.0s` regardless of turn number.

5.4 Thinking Token Volatility & 2D Trajectory Snapshots (Turn 1 vs Turn 25 vs Turn 50)

To prove what drives latency under `thinking=default` and visualize how chunks arrive across early (**Turn 1**), mid (**Turn 25**), and late (**Turn 50**) conversation turns, Figure 5-4 pairs a **scatter plot of** `thoughts_tokens` **vs** `total_ms` **across all 100** `thinking=default` **executions (** `50 turns × 2 modes` **)** on the left with a **3-Turn Trajectory Overlay (** `Turn 1` **,** `Turn 25` **,** `Turn 50` **from** `load_multiturn_trajectory()` **)** on the right.

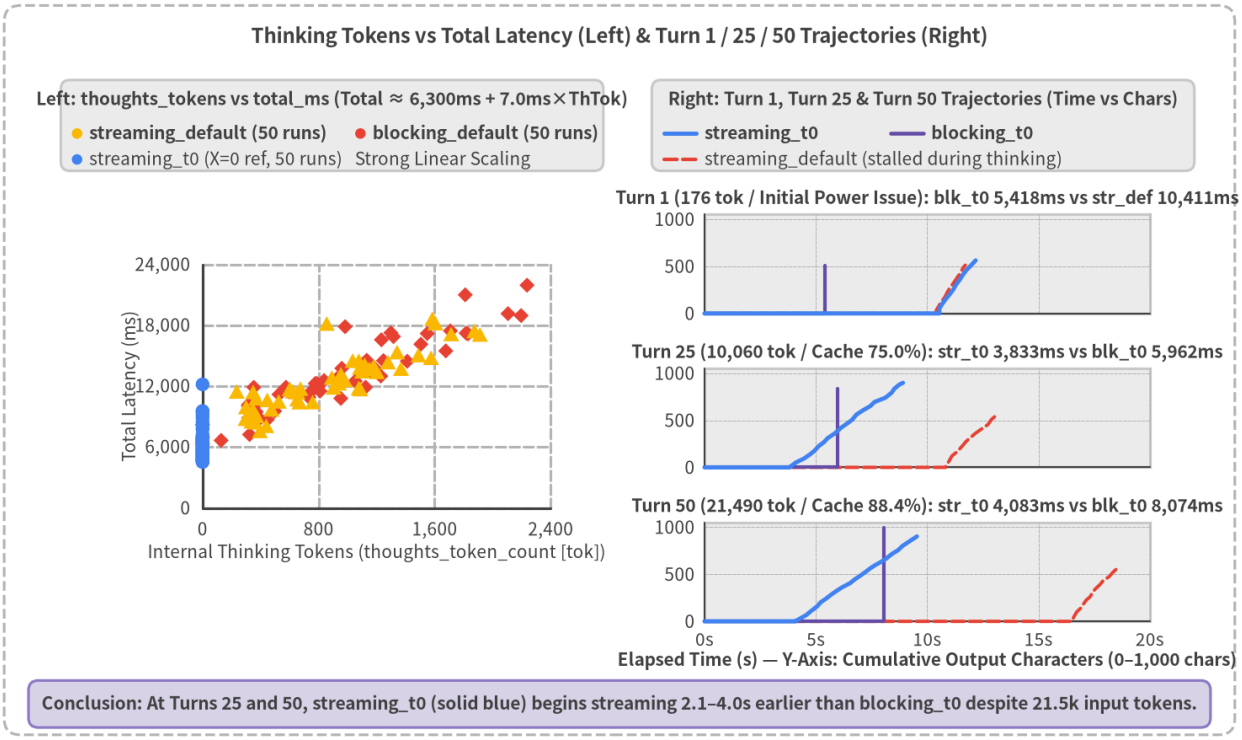


Figure 5-4: Thinking Tokens vs Total Latency Scatter Plot (Left) & 3-Turn Stream Trajectory Overlay at Turns 1, 25, and 50 (Right)

Key Findings from the Scatter & Trajectory Analysis

1. **Left Panel: `Total ≈ 6,300 ms + 7.0 ms × ThTok` Governs Multi-Turn Latency**:
   - Plotting all 100 `thinking=default` executions (`50 turns × 2 modes`) confirms that total completion time is governed by reasoning token count (`0–2,237 tokens`), **not** turn number (`Total ≈ 6,300 ms + 7.0 ms × ThTok`).
   - The `~6,300 ms` y-intercept represents the base decoding time for a `~753-char` detailed technical support response, while each 1,000 thinking tokens add `~7.0 seconds` of pre-output delay.
2. **Right Panel: 3-Turn Trajectory Snapshots (`Turn 1` , `Turn 25` , `Turn 50`)**:
   - **Turn 1 (`176 tok` , Cache `0%` , Initial Unresponsive PC Query)**: Apart from the initial cold request, `blocking_t0` (solid purple) remains flat at `0 chars` until `5,418 ms` , and `streaming_default` (dashed red) stalls at `0 chars` for `10,411 ms` (`1,087` thinking tokens) before streaming.
   - **Turn 25 (`10,060 tok` , Cache `7,547 tok [75.0%]` , BIOS/Chipset Update Guide)**: Even with 57× more input context, `streaming_t0` (solid blue) starts streaming at

`3,833 ms`, delivering the first visible characters `2,129 ms` (`36%`) **earlier** than `blocking_t0` (`5,962 ms`).

- **Turn 50 (`21,490 tok` , Cache `18,995 tok [88.4%]` , Preventive Maintenance Summary)**: At the 50th turn (`122×` larger context), Implicit Context Caching serves `88.4–90.6%` of the prompt from KV cache. As a result, `streaming_t0` begins streaming at `4,083 ms` — `3,991 ms` (`49%`) **earlier** than `blocking_t0` (`8,074 ms`)— and completes the `902-char` reply at `9,554 ms` (whereas `streaming_default` stalls for `16,498 ms` due to `1,584` thinking tokens).

5.5 Comprehensive 10-Turn Cohort Table & Production Guidelines for Multi-Turn Chat

1. 10-Turn Cohort Summary Table (`Turns 1–10` through `Turns 41–50`)

The table below summarizes input token growth, Implicit Context Cache hit rates, and latency across 10-turn cohorts and all 50 turns combined.

Turn Cohort & Troubleshooting Stage	Avg `prompt_tokens` (Min–Max)	Avg `cached_tokens` (`streaming_t0` Hit Rate)	`streaming_t0` TTFT / Total (`ms`)	`blocking_t0` TTFT = Total (`ms`)	Streaming TTFT Lead (`streaming_t0` - `blocking_t0`)	`streaming_` TTFT / Tot [Avg
Turns 1–10 (Power, Discharge, BIOS, Safe Mode)	`1,768 tok` (`176–3,407`)	`177 tok` (`10.0%` , active T8–9)	`2,853` / `5,910 ms` (T2–10: `2,000 ms`)	`5,125 ms`	`-2,272 ms` (`-44.3%`)	`11,687` / [`1,2`
Turns 11–20 (Backup, Drivers, `sfc` , Memory Diag)	`5,635 tok` (`3,817–7,539`)	`3,229 tok` (`57.3%`)	`2,180` / `5,586 ms`	`6,742 ms`	`-4,562 ms` (`-67.7%`)	`9,357` / [`8`
Turns 21–30 (SSD SMART, Restore, USB Boot Media)	`10,280 tok` (`8,005–12,478`)	`7,784 tok` (`75.7%`)	`2,361` / `6,804 ms`	`6,666 ms`	`-4,305 ms` (`-64.6%`)	`9,969` / [`9`

Turn Cohort & Troubleshooting Stage	Avg `prompt_tokens` (Min–Max)	Avg `cached_tokens` (`streaming_t0` Hit Rate)	`streaming_t0` TTFT / Total (`ms`)	`blocking_t0` TTFT = Total (`ms`)	Streaming TTFT Lead (`streaming_t0` - `blocking_t0`)	`streaming_` TTFT / Tot [Avg
Turns 31–40 (BitLocker, HW Diag, Pickup Repair)	`15,007 tok` (`12,995–16,915`)	`12,556 tok` (`83.7%`)	`2,331` / `6,260 ms`	`6,653 ms`	`-4,322 ms` (`-65.0%`)	`8,265` / [`5`
Turns 41–50 (Loaner PC, On-Site, Backup & Maint)	`19,338 tok` (`17,309–21,490`)	`16,872 tok` (`87.2%`)	`2,574` / `7,054 ms`	`6,935 ms`	`-4,361 ms` (`-62.9%`)	`9,910` / [`8`
All 50 Turns	`10,406 tok` (`176–21,490`)	`8,124 tok` (`78.1%` / Peak `90.6%`)	`2,460` / `6,323 ms` (Median `2,029 ms`)	`6,424 ms`	`-3,964 ms` (`-61.7%`)	`9,838` / [`8`

2. Production Architecture Guidelines for Multi-Turn Chat

1. **Preserve Strict Prefix Immutability to Maximize Implicit Context Caching**:
   - Gemini 3.8 Flash's Implicit Context Caching matches the exact token prefix across `system_instruction` and historical `contents: list[types.Content]`.
   - **Anti-Pattern**: Injecting per-request dynamic timestamps, random request UUIDs, or mutating device telemetry JSON inside `system_instruction` invalidates the token prefix on every turn, dropping the cache hit rate to `0%`.
   - **Best Practice**: Keep `system_instruction` static and treat `history: list[types.Content]` as strictly **append-only**. Place dynamic per-turn metadata (such as current diagnostic status or retrieved knowledge-base snippets) at the end of the latest user turn message.
2. **Standardize on `Streaming × thinking=0` (`streaming_t0`) for Conversational UX**:
   - When technical support responses average `~753 chars`, full token generation takes `~6.42 seconds` even with `thinking=0`.
   - Using `generate_content_stream` with `thinking_budget=0` delivers the first token in `2,460 ms` **on average (** `2,029 ms` **median,** `3,964 ms` **/** `62%` **earlier than Blocking)** and scales at just `+29 ms per 1,000 input tokens`, maintaining responsive UX through Turn 50.

3. **Combine Multi-Turn Prefix Caching with the Two-Field Integrated Adaptive Router (Pattern A)**:
    - Leaving `thinking=default` enabled across all turns inflates average TTFT to `9,838 ms`.
    - Instead, pair `thinking=0` streaming for routine troubleshooting turns with the **2-Field Integrated Pydantic Router (Pattern A)** from Chapter 4 when a turn requires complex multi-step cost or transfer-time math. Because Stage 1 (`thinking=0` prefix check) and Stage 2 (`thinking=low` parallel handoff) share the exact same `system_instruction` + `history` prefix, Stage 2 automatically benefits from the warmed Implicit Context Cache.